UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Immagina... scarichi un file MP3, registrato negli USA, cantato da un francese, memorizzato in un server giapponese, spedito su Internet attraverso l'Asia e l'Europa ed ascoltato sulla cima di una montagna austriaca su un cellulare prodotto in Germania: com'è possibile tutto ciò?

CORSO DI LAUREA IN INFORMATICA

INSEGNAMENTO DI INGEGNERIA DEL SOFTWARE

A.A. 2025-2026

Progettazione e Sviluppo del Sistema BugBoard26

A cura di

Antonio Esposito
N86005375
antonio.esposito167@studenti.unina.it

Vincenzo Di Carluccio
N86004800
vi.dicarluccio@studenti.unina.it

Docenti

Sergio Di Martino Luigi Libero Lucio Starace

Sommario Generale

La documentazione del progetto è suddivisa nei seguenti 4 documenti, tutti indipendenti tra loro e rappresentanti ciascuna delle fasi chiave della progettazione e implementazione del sistema.

1. Documento di Specifica dei Requisiti Software ……

2. Documento Di System Design ……

3. Documento di Design del Software ……

4. Documento di Software Testing ……

DOCUMENTO DI SPECIFICA DEI REQUISITI SOFTWARE

BugBoard26

ID Documento	BB26-001
Versione	1.0
Stato	Rilascio Iniziale
Data di Rilascio	10/02/2026

Progetto di Ingegneria del Software – A.A. 2025/2026

Redatto da:

Antonio Esposito

Vincenzo Di Carluccio

Sommario

0. Introduzione 6
0.1 Scopo del Sistema 6
0.2 Contesto Operativo 6
0.2.1 Scenario Tipico di Utilizzo 6
0.2.2 Scopo del sistema 7
0.3 Assunzioni e Vincoli 7
0.4 Stakeholder 8
0.5 Organizzazione del Documento 8
1. Glossario 9
1.1 Termini e Definizioni 9
2. Casi D'uso 12
2.1 Diagramma dei Casi D'uso 13
2.2 UC-01: Autenticazione 14
2.3 UC-02: Creazione Utenza 14
2.4 UC-03: Segnalazione nuova Issue 14
2.4.1 Tabella Cockburn UC-03 15
2.5 UC-04: Visualizzazione lista Issue 18
2.5.1 Tabella Cockburn UC-04 18
2.6 UC-05: Aggiunta commento ad Issue 21
2.6.1 Tabella Cockburn UC-05 21
2.7 UC-06: Modificare Issue 24
2.8 UC-07: Visualizzazione dettagli Issue 24
2.9 UC-08: Effettuare il Logout 24
2.10 Matrice delle autorizzazioni 25
3. Personas 26

3.1 Obiettivi e Valore Strategico delle Personas....26
3.2 Schede Persona....27
4. Requisiti Non Funzionali e di Dominio....30
4.1 Requisiti Non Funzionali....30
4.1.1 Usabilità....30
4.1.2 Performance....30
4.1.3 Sicurezza....30
4.1.5 Manutenibilità....31
4.1.6 Compatibilità....31
4.2 Requisiti di Dominio....32

0. Introduzione

0.1 Scopo del Sistema

BugBoard26 è una piattaforma web collaborativa progettata per gestire in modo centralizzato il ciclo di vita delle issue all'interno di team di sviluppo software.

BugBoard26 si concentra specificamente sulla gestione delle segnalazioni tecniche, offrendo un ambiente strutturato dove sviluppatori, project manager e stakeholder non tecnici possono collaborare efficacemente. Il sistema permette di:

- **Tracciare anomalie software** (bug) con descrizioni dettagliate e allegati visivi
- **Segnalare lacune documentali** che necessitano integrazione
- **Porre domande tecniche** al team in modo asincrono
- **Monitorare lo stato di avanzamento** attraverso dashboard riepilogative
- **Assegnare responsabilità** in modo chiaro e tracciabile

L'obiettivo primario è **ridurre la dispersione informativa** che tipicamente affligge i progetti software, dove le segnalazioni viaggiano su canali eterogenei (email, messaggistica, verbali di riunione) rischiando di perdersi o duplicarsi. BugBoard26 centralizza tali informazioni in un unico sistema.

0.2 Contesto Operativo

Il sistema si inserisce nel contesto **di software houses e team di sviluppo** di piccole dimesioni, che necessitano di uno strumento leggero ma efficace per la gestione quotidiana delle issue.

0.2.1 Scenario Tipico di Utilizzo

Una software house sviluppa applicazioni web per clienti. Il team è composto da:

- 1 Project Manager che coordina 10 sviluppatori
- Sviluppatori junior e senior che lavorano in modalità ibrida
- Stakeholder non tecnici che necessitano visibilità sullo stato del progetto senza dover interpretare tool complessi

0.2.2 Scopo del sistema

Problematiche Attuali:

- Le segnalazioni arrivano via messagi, email, chiamate → rischio di perdita
- Nessuna prioritizzazione chiara → sviluppatori confusi su cosa affrontare
- Gli stakeholder non hanno visibilità autonoma → bombardano il PM di richieste di aggiornamento
- Manca storicizzazione delle decisioni tecniche → difficile ricordare "perché è stato fatto così"

BugBoard26 come Soluzione: Il sistema fornisce un canale ufficiale e strutturato dove ogni segnalazione viene registrata, classificata, assegnata e tracciata fino alla risoluzione. Gli sviluppatori sanno sempre cosa fare, il PM ha visione d'insieme, gli stakeholder consultano autonomamente lo stato senza interrompere il team.

0.3 Assunzioni e Vincoli

Assunzioni:

- Gli utenti hanno accesso a browser moderni e relativamente aggiornati.
- Tutti gli utenti hanno una connessione ad internet con larghezza di banda nella media e stabile.

Vincoli:

- **Budget:** Progetto accademico, tecnologie open-source, nessun costo.
- **Team:** 2 studenti, architettura semplice.

Si cercherà di approcciare il problema in modo più professionale possibile e coerente coi principi dell’ingegneria del software, senza però sfociare in over-engineering o perdite di tempo inutili durante l’implementazione.

0.4 Stakeholder

Stakeholder	Ruolo	Interesse Primario
A. Esposito **V. Di Carluccio**	Team di sviluppo	Implementare sistema manutenibile e tecnicamente valido secondo i canoni dell'ingegneria del software
L.L.L. Starace **S. Di Martino**	Professore Universitario	Valutare competenze in ambito ingegneria del software e fornire una valutazione sulla base del lavoro fatto
S. Esposito (Persona)	Project Manager	Coordinamento team e monitoraggio avanzamento
A. Romano (Persona)	Sviluppatore Junior	Produttività personale e chiarezza assegnazioni
C. Martinelli (Persona)	Stakeholder Non-Tecnico	Visibilità stato progetto senza complessità tecnica

0.5 Organizzazione del Documento

Il presente documento è strutturato come segue:

- **Capitolo 1** - Glossario**:** Definizioni terminologiche per evitare ambiguità interpretative.
- **Capitolo 2** - Casi D'uso: Diagrammi UML, descrizioni funzionali, tabelle Cockburn e mockup per requisiti funzionali.
- **Capitolo 3** - Personas**:** Archetipi utenti per guidare decisioni di design.
- **Capitolo 4** - Requisiti Non Funzionali e di Dominio: Vincoli di qualità, performance, sicurezza e regole di business.

1. Glossario

In questa sezione vengono definiti i termini tecnici e gli acronimi di dominio utilizzati all'interno del documento di specifica.

L'obiettivo di questo glossario è stabilire un linguaggio uniforme condiviso tra il team di sviluppo e gli stakeholder non tecnici. Ogni termine qui riportato ha un significato specifico e vincolante per il progetto, l'uso coerente di questa terminologia è fondamentale per evitare ambiguità interpretative che potrebbero tradursi in errori durante la fase di implementazione.

Si raccomanda di fare riferimento alle definizioni seguenti ogni volta che, nel corso della lettura, si incontrano termini specifici non immediatamente chiari.

1.1 Termini e Definizioni

Termine	Definizione
Requisito Funzionale	Un servizio offerto dal sistema all'utente, il modo in cui il sistema reagisce a determinati input.
Requisito Non Funzionale	Vincoli che il sistema deve rispettare per offrire i servizi
Requisito di Dominio	Vincolo o funzionalità che deriva direttamente dal contesto operativo, normativo o professionale in cui il sistema agisce (es. leggi fiscali, standard medici, regole matematiche). A differenza dei requisiti funzionali, non sono "richiesti" dall'utente ma sono intrinseci al settore di riferimento.
UML	Unified Modeling Language. Linguaggio standardizzato di modellazione visiva utilizzato per specificare, visualizzare, costruire e documentare gli artefatti di un sistema software.
Attore	Entità (non necessariamente umana) che interagisce col sistema richiedendone i servizi. In UML è rappresentato da un omino stilizzato.
Caso d'uso	Una specifica unità di funzionalità del sistema che permette a un Attore di raggiungere un obiettivo prefissato. Descrive la sequenza di interazioni tra l'utente e il software che servono ad

	implementare il requisito funzionale. In UML rappresentato da un ovale
Associazione	In UML, la linea che collega un Attore a un Caso d'Uso. Indica che l'attore partecipa ad una funzionalità. Non ha direzione per indicare il flusso di comunicazione.
Generalizzazione	Relazione di ereditarietà (freccia con triangolo vuoto). Indica che un Attore o un Caso d'Uso è una versione specifica di uno più generico. Quello più specifico eredita tutte le caratteristiche e comportamenti di quello più generale.
Inclusione (UML) <<include>>	Relazione obbligatoria. Indica che il Caso d'Uso A necessita dell'esecuzione del Caso d'Uso B per essere completato. La freccia tratteggiata punta verso il caso incluso.
Estensione (UML) <<extend>>	Relazione opzionale tra due Casi d'Uso. Indica che il Caso d'Uso A potrebbe essere eseguito durante l'esecuzione del Caso d'Uso B, ma solo a certe condizioni. La freccia tratteggiata punta verso il caso base
Scenario	Una specifica sequenza di azioni e interazioni tra gli attori e il sistema. Può essere il percorso principale o una variante.
Tabella Cockburn	Formato strutturato per la descrizione testuale dei Casi d'Uso, ideato da Alistair Cockburn. Prevede sezioni specifiche per scenario principale, estensioni e trigger.
Precondizione	Requisito che deve essere vero prima che un Caso d'Uso possa iniziare.
Post-condizione di successo	Lo stato in cui si trova il sistema dopo che il Caso d'Uso è stato completato con successo.
Post-condizione di fallimento	Lo stato in cui si trova il sistema se il caso d'uso non ha avuto successo.
Trigger	Azione (eseguita di solito da un Attore) che da inizio ad un caso d'uso.
Estensione (Cockburn)	Percorso alternativo o una gestione di errore che devia dal Flusso Principale.

Persona	Personaggio fittizio creato per rappresentare un segmento di utenti target. Include dati demografici, obiettivi e Bio. Serve a guidare le decisioni di design.
Mockup	Rappresentazione approssimativa dell'interfaccia utente. Mostra il layout e i contenuti delle schermate senza implementare la logica reale o la grafica definitiva.
Stakeholder	Chiunque abbia un interesse (economico o meno) nel successo del sistema, anche se non interagisce direttamente con esso. Le **Personas** servono ad esplicitare i loro interessi.
Issue	Termine generico che indica un problema o richiesta relativo ad un software.
Bug	Un'anomalia o un difetto del software che causa un comportamento inatteso o errato, impedendo al sistema di soddisfare i requisiti specificati o portandolo a un arresto anomalo.
Feature	Una nuova funzionalità o caratteristica del software che aggiunge valore all'utente. Non è una correzione di un errore, ma un'espansione delle capacità del sistema.
UI	User Interface: l'interfaccia utente che consente agli utenti finali di comunicare col software.

2. Casi D'uso

In questo capitolo vengono definiti e formalizzati i requisiti funzionali del sistema BugBoard26. L'analisi si concentra sulla descrizione del comportamento atteso del software, delineando le modalità con cui gli attori (utenti e amministratori) interagiscono con la piattaforma per raggiungere i propri obiettivi.

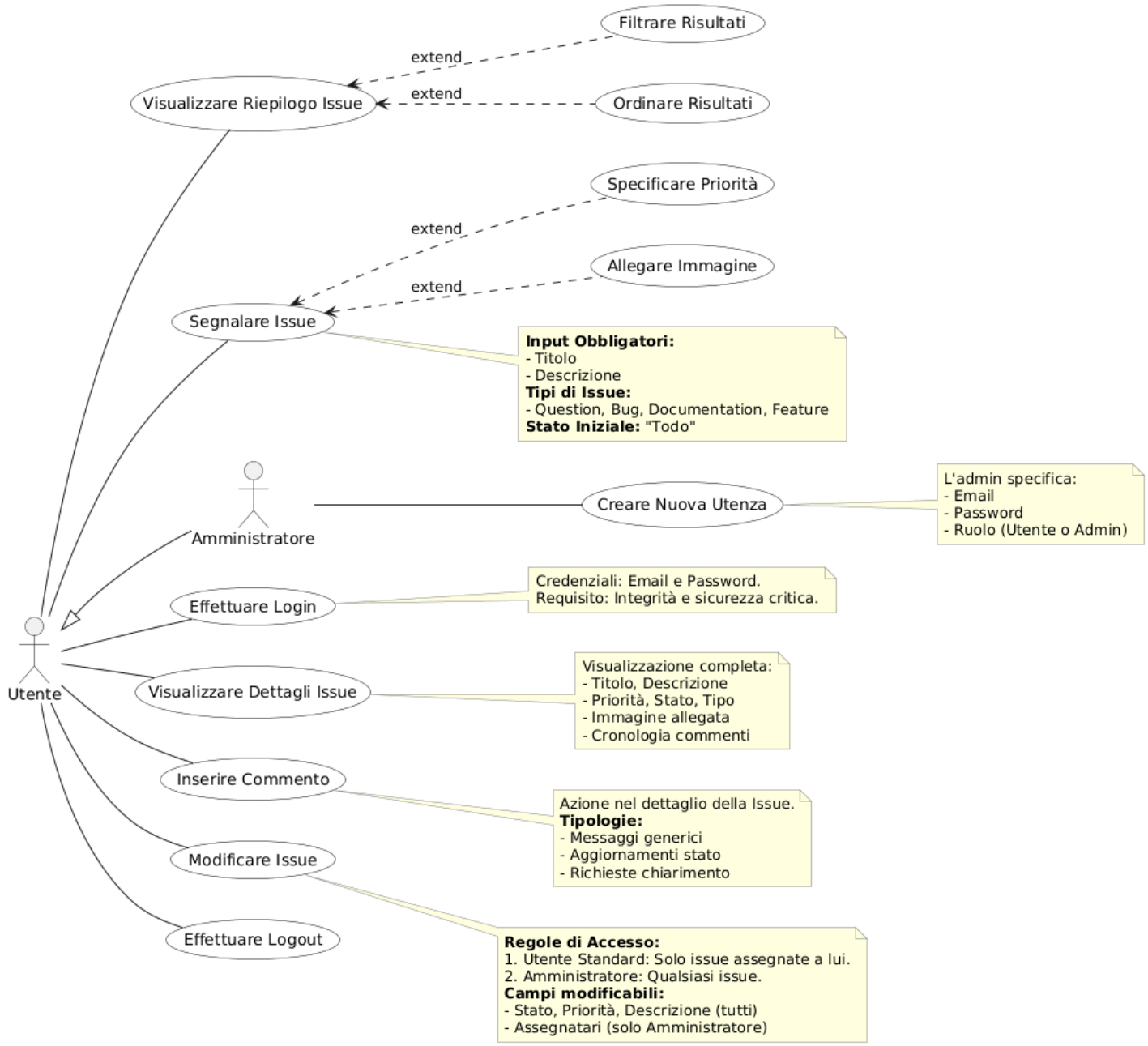
L'obiettivo è tradurre le specifiche iniziali in modelli strutturati che fungano da riferimento univoco per lo sviluppo dell'architettura distribuita. Verranno quindi esplorati i confini del sistema, le logiche di business e le interfacce utente previste.

Nello specifico, il capitolo è strutturato nei seguenti punti:

- **Diagrammi dei Casi D'uso:** Verrà utilizzato lo standard UML per fornire una rappresentazione grafica ad alto livello delle funzionalità. Questi diagrammi permettono di identificare immediatamente gli attori coinvolti e le relazioni che intercorrono tra loro e i servizi offerti dal sistema, offrendo una visione d'insieme chiara e accessibile anche ai non tecnici.
- **Descrizioni Funzionali:** A supporto della rappresentazione grafica, ogni caso d'uso sarà accompagnato da una descrizione testuale sintetica.
- **Tabelle di Cockburn e Mockup:** Per i casi d'uso non banali o che rappresentano funzionalità critiche offerte dal sistema, vengono forniti anche i seguenti artefatti:
    - **Tabelle Cockburn:** Una formalizzazione rigorosa che dettaglia non solo il flusso principale, ma anche gli scenari alternativi ed estensioni, garantendo che ogni possibile variante dell'interazione sia stata prevista.
    - **Mockup dell'interfaccia:** Bozze visuali delle schermate. Questi prototipi grafici sono essenziali per mostrare come i requisiti funzionali verranno tradotti visivamente nell'applicazione web, facilitando la comprensione del flusso operativo.

Di seguito lo Use case diagram del sistema, si specifica che l'ordine col quale vengono riportati i casi d'uso è del tutto irrilevante e non sottintende un qualsiasi tipo di gerarchia.

2.1 Diagramma dei Casi D'uso



2.2 UC-01: Autenticazione

Questo caso d'uso rappresenta il punto di ingresso obbligatorio alla piattaforma. Data la natura critica delle informazioni gestite, l'accesso è strettamente regolato per garantire l'integrità e la riservatezza dei dati.

Il sistema richiede l'inserimento di Email e Password. Solo a seguito di una verifica positiva della corrispondenza di tali dati nel sistema di persistenza, l'attore ottiene l'accesso alle funzionalità operative della dashboard.

2.3 UC-02: Creazione Utenza

Questo caso d'uso definisce la procedura di creazione per le utenze del sistema. Poiché il sistema non prevede una registrazione pubblica autonoma, la creazione degli account è una funzionalità critica delegata esclusivamente agli amministratori.

Il diagramma mostra l'interazione esclusiva dell'attore Amministratore. Partendo dall'account di default o da un account con privilegi elevati, l'Amministratore inserisce nel sistema i dati necessari per registrare un nuovo utente. Durante questa operazione, è richiesto di specificare una mail univoca e una password. Infine, l'amministratore decide se la nuova utenza avrà privilegi standard o se godrà a sua volta di permessi amministrativi.

2.4 UC-03: Segnalazione nuova Issue

La segnalazione di una issue rappresenta il cuore di BugBoard26. Attraverso questa funzionalità, gli utenti alimentano il flusso di lavoro, trasformando problemi, idee o necessità di documentazione in elementi tracciabili e gestibili.

L'attore (Utente, e di conseguenza Amministratore) avvia la creazione di una nuova Issue nel sistema. È obbligatorio fornire un titolo e una descrizione. Il sistema richiede di classificare la segnalazione in una delle tipologie predefinite (Question, Bug, Documentation, Feature) e imposta automaticamente lo stato iniziale su "**todo**". Il processo prevede due estensioni facoltative: l'utente può decidere di Specificare la priorità e/o Allegare un'Immagine (screenshot o diagramma) per chiarire meglio il contesto del problema.

La nuova issue creata viene automaticamente assegnata a chi l'ha creata.

2.4.1 Tabella Cockburn UC-03

USE CASE #03	Segnalazione nuova Issue	
Obiettivo	Un utente autenticato vuole segnalare una nuova issue nel sistema	
Precondizioni	L'utente deve aver completato UC-01	
Post-condizione di successo	La nuova issue con stato "todo" è stata creata e salvata nel database	
Post-condizione di fallimento	Nessuna issue viene creata.	
Attori principali	Utente Autenticato	
Trigger	L'utente seleziona l'opzione "Nuova Issue" dall'interfaccia	
Step n.	**Utente**	**Sistema**
1	Clicca su "Nuova Issue"	
2		Mostra M2
3	Compila i campi relativi all'issue	
4	Clicca su "Crea Issue"	
5		Mostra M3
6	Clicca su "Conferma"	
7		Valida i dati inseriti
8		Salva l'issue con stato "todo"
9		Mostra M4
Estensione #1 - Campo mancante		
Step n.	**Utente Autenticato**	**Sistema**
7a	Almeno un campo obbligatorio è lasciato vuoto	

8a		Evidenzia il campo vuoto in rosso
9a		Scrive: "Il campo x è obbligatorio" nella parte bassa del form
10a	Torna allo step 3	
Estensione #2 - File immagine non valido		
Step n.	**Utente Autenticato**	**Sistema**
3b	Allega file con formato non supportato nella sezione immagine	
4b		Mostra M5
5b	Torna allo step 3	
Estensione #3 - Errore generico creazione issue		
Step n.	**Utente Autenticato**	**Sistema**
8c		Rileva un errore durante il salvataggio della issue
9c		Mostra M5
10c	Click su “OK”	
11c		Mostra M1

Figura 2.3.1: Mockup UC-03

2.5 UC-04: Visualizzazione lista Issue

Per garantire un monitoraggio efficace dell'avanzamento di un progetto, il sistema offre una dashboard riepilogativa delle issue.

L'attore accede alla lista generale delle issue per consultarne lo stato corrente. Il sistema presenta i dati in una vista sintetica. Per facilitare l'analisi in presenza di numerose segnalazioni, sono previste due funzionalità estese opzionali: l'utente può Filtrare i risultati applicando criteri specifici, oppure Ordinare la lista secondo data o priorità della stessa.

2.5.1 Tabella Cockburn UC-04

USE CASE #04	Visualizzazione Riepilogativa Issue	
Obiettivo	Un utente autenticato vuole visualizzare l'elenco delle issue, con possibilità di filtrare e ordinare i risultati per trovare rapidamente le issue di suo interesse	
Precondizioni	L'utente è autenticato nel sistema	
Post-condizione di successo	L'utente visualizza l'elenco delle issue filtrato/ordinato secondo i criteri selezionati	
Post-condizione di fallimento	L'elenco non viene mostrato o i filtri non vengono applicati	
Attori principali	Utente Autenticato	
Trigger	L'utente accede alla sezione "Issue" del sistema	
Step n.	**Utente**	**Sistema**
1	Dalla dashboard principale, seleziona un progetto, di default il primo nella lista	
2		Carica l'elenco di tutte le issue dal database relative ad un progetto
3		Mostra M1
4	Clicca su filtri	
5		Mostra M2

Estensione #1 - Nessuna issue presente nel sistema		
Step n.	**Utente Autenticato**	**Sistema**
2a		Non trova issue nel database
3a		Mostra M3
Estensione #2 - Nessuna issue corrisponde ai filtri		
Step n.	**Utente Autenticato**	**Sistema**
5b		Mostra M4
6b	Torna allo step 4	
Estensione #3 - Errore di caricamento dati		
Step n.	**Utente Autenticato**	**Sistema**
2c		Rileva un errore durante il caricamento dei dati in memoria
3c		Mostra M5

Figura 2.5.1: Mockup UC-04

2.6 UC-05: Aggiunta commento ad Issue

La collaborazione efficace in BugBoard26 non si limita alla creazione delle segnalazioni, ma prosegue attraverso la comunicazione continua. Questo caso d'uso definisce il meccanismo di feedback e aggiornamento asincrono tra i membri del team.

L'attore (**Utente** o **Amministratore**) interagisce con una issue esistente per aggiungere informazioni. Questa funzionalità è cruciale per tracciare l'evoluzione della risoluzione, richiedere chiarimenti tecnici o fornire aggiornamenti di stato senza dover modificare la descrizione originale del problema.

2.6.1 Tabella Cockburn UC-05

USE CASE #05	Aggiunta Commento a Issue	
Obiettivo	Un utente autenticato vuole aggiungere un commento a una issue esistente per fornire aggiornamenti o chiedere chiarimenti	
Precondizioni	L'utente è autenticato, esiste almeno una issue nel sistema	
Post-condizione di successo	Il commento viene salvato nel database ed è visibile a tutti gli utenti	
Post-condizione di fallimento	Nessun commento viene aggiunto	
Attori principali	Utente Autenticato	
Trigger	L'utente clicca su una issue	
Step n.	**Utente**	**Sistema**
1	Clicca sulla issue nella quale desidera aggiungere un commento	
2		Mostra M2
3	Inserisce il testo relativo al commento	
4	Clicca su “Aggiungi Commento”	
5		Salva commento nel database
6		Mostra M5

7		Torna allo step 2
Estensione #1 - Errore nel caricamento dei commenti		
Step n.	**Utente**	**Sistema**
2a		Rileva errore nel caricare in memoria i commenti
3a		Mostra M6
4a	Clicca su “OK”	
5a		Mostra M1
Estensione #2 - Errore nel salvataggio del commento		
Step n.	**Utente**	**Sistema**
6b		Rileva che c’è stato un errore durante il salvataggio del commento nel database
7b		Mostra M6
8b	Clicca su “OK”	
9b		Torna allo step 2

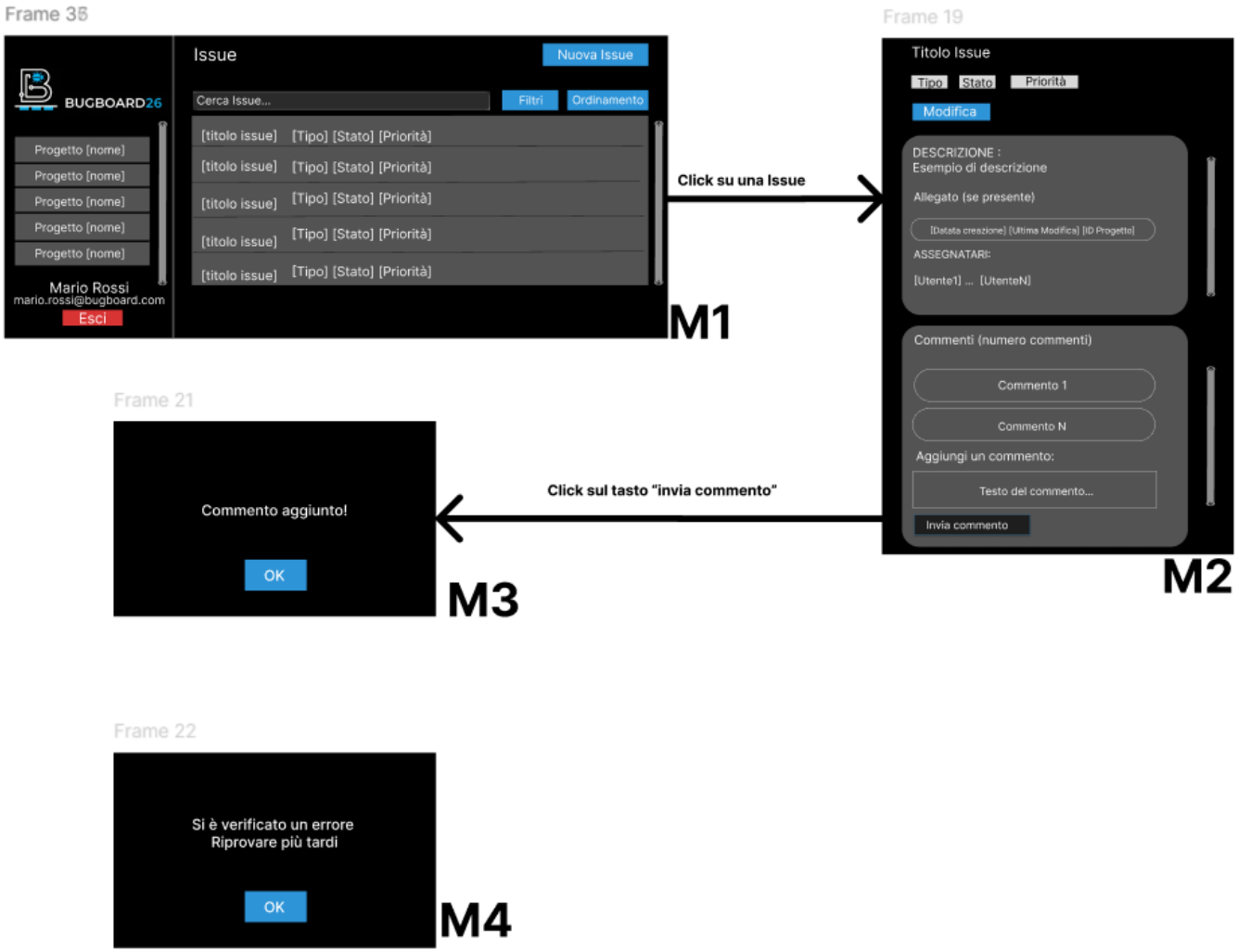


Figura 2.6.1: Mockup UC-05

2.7 UC-06: Modificare Issue

Le issue richiedono aggiornamenti costanti durante il ciclo di vita del progetto. Questo caso d'uso permette di alterare le informazioni di una issue esistente, ossia descrizione, stato, priorità e persone a cui la issue è assegnata per riflettere i cambiamenti avvenuti.

Il sistema applica una rigida politica di controllo degli accessi per questa operazione: L'Amministratore ha il controllo completo e può intervenire su qualsiasi segnalazione, l'Utente Standard ha permessi limitati e può modificare esclusivamente le issue che gli sono state assegnate direttamente.

2.8 UC-07: Visualizzazione dettagli Issue

Per mantenere una dashboard principale il più pulita e sintetica possibile, nella sezione dedicata alle Issue vengono riportate soltanto le informazioni principali relative a quest'ultime, ossia titolo, stato, priorità e la data di creazione.

La visualizzazione completa dei dettagli di una Issue include anche descrizione, autore e data relativa all'ultima modifica.

2.9 UC-08: Effettuare il Logout

È previsto un pulsante cliccabile sulla dashboard principale del sistema per effettuare il logout. Alla pressione del pulsante si viene riportati alla schermata di login.

2.10 Matrice delle autorizzazioni

Caso D'uso	Utente Standard	Amministratore
UC-01: Autenticazione	Può autenticarsi con le proprie credenziali	Può autenticarsi con le proprie credenziali
UC-02: Creazione Utenza	NON può creare nuovi utenti	Può creare nuovi utenti e assegnare ruoli (Utente o Admin)
UC-03: Segnalazione nuova Issue	Può creare issue nei progetti a cui appartiene. L'issue viene automaticamente assegnata a sé stesso	Può creare issue in qualsiasi progetto del sistema
UC-04: Visualizzazione lista Issue	Visualizza solo le issue a cui è assegnato	Visualizza tutte le issue nel sistema
UC-05: Aggiunta commento ad Issue	Può commentare tutte le sue issue visibili	Può commentare qualsiasi issue del sistema
UC-06: Modificare Issue	Può modificare solo le issue che gli sono assegnate	Può modificare qualsiasi issue del sistema e modificare gli assegnatari di quest'ultime
UC-07: Visualizzazione dettagli Issue	Può visualizzare i dettagli completi delle issue che gli sono asegnate	Può visualizzare i dettagli completi di qualsiasi issue
UC-08: Effettuare Logout	Può effettuare il logout premendo un pulsante nella dashboard principale	Può effettuare il logout premendo un pulsante nella dashboard principale

3. Personas

In questa sezione vengono definiti archetipi degli utenti che interagiranno con il sistema. Le persone qui descritte non sono individui reali, ma rappresentazioni puramente fittizie di prototipi di persone che andranno ad interagire col sistema. Esse sintetizzano i comportamenti, le motivazioni e gli obiettivi dell'utenza target.

3.1 Obiettivi e Valore Strategico delle Personas

L'integrazione delle Personas nel processo di sviluppo è fondamentale per garantire che il software risponda a problemi reali. Nello specifico, servono a:

- **Guidare le decisioni di design**: L'interfaccia del sistema deve essere intuitiva e reattiva, ogni utente, a prescindere dal suo background tecnico, deve essere in grado di interfacciarsi efficacemente col sistema.
- **Ridurre le ambiguità dei requisiti**: Spostano il focus dalle funzionalità astratte ("Il sistema deve…") ai bisogni concreti ("Persona X ha bisogno di un modo semplice per segnalare problemi del software").
- **Prioritizzare lo sviluppo**: Aiutano a distinguere le funzionalità critiche da quelle secondarie in base all'impatto che hanno sugli obiettivi primari dell'utente.
- **Focalizzare il Team**: Creano più familiarità tra il team di sviluppo e gli stakeholder, impedendo che il team si focalizzi troppo sulle proprie preferenze durante lo sviluppo.

Ogni scheda Persona che segue include dati demografici, competenze tecniche, e gli obiettivi che il nuovo sistema deve aiutare a raggiungere, insieme ad una breve biografia della Persona, in modo da renderla più "reale".

3.2 Schede Persona

Sofia Esposito

Age: 40

Marital status: Sposata con 2 figli

Location: Napoli, Italia

Job title: Project Manager

Personal traits: Estremamente organizzata, autoritaria, orientata ai risultati, comunicativa

Obiettivi:

- Coordinare efficacemente il team di sviluppo assegnando i bug e le feature da implementare equamente
- Monitorare in ogni momento lo stato del progetto tramite la funzionalità di vista riepilogativa delle issue
- Garantire che le issue critiche vengano risolte rispettando le scadenze
- Generare report per i propri stakeholder

Interessi:

- **Fitness:** Le piace staccare dalla vita di ufficio per fare sport
- **Team Building:** Organizza attività extra-lavorative per migliorare la coesione del team
- **Serie TV:** Le piacciono le serie TV, in particolare Brooklyn 99 e Stranger Things
- **Continuous Improvement:** Sempre alla ricerca di modi per migliorarsi

Bio:

Sofia è Project Manager da 3 anni presso una software house napoletana con un team di 10 sviluppatori. Ha iniziato come sviluppatrice e nel tempo è cresciuta fino ad assumere responsabilità manageriali. Ora il suo focus principale è la gestione del team e la qualità del processo di sviluppo. Il suo obiettivo è far sì che il team lavori in modo fluido, identificando e risolvendo rapidamente eventuali problemi organizzativi. Deve bilanciare le esigenze tecniche del team con le pressioni del management per rispettare le deadline.

Carla Martinelli

Age: 54

Marital status: Divorziata

Location: Milano, Italia

Job title: Direttrice Marketing Pubblicitario

Personal traits: Impaziente, esigente, comunicazione diretta, poche conoscenze informatiche

Obiettivi:

- Controllare autonomamente lo stato delle issue senza dover contattare il project manager
- Monitorare i tempi di risoluzione dei problemi per assicurarsi che lo sviluppo proceda senza intoppi
- Ottenere una visione di alto livello dello stato del progetto senza entrare nei dettagli tecnici
- Prendere decisioni informate su investimenti futuri nel progetto

Interessi:

- **Insegnamento:** Mentoring di giovani professionisti
- **Arte contemporanea:** Colleziona opere d'arte nel tempo libero
- **Viaggi:** Ama viaggiare per il mondo per fare nuove esperienze
- **Semplicità:** Ama le cose semplici e intuitive

Bio:

Carla è da 10 anni direttrice di un importante gruppo pubblicitario con sede a Milano. Ha iniziato la sua carriera come stagista nella stessa azienda, da quando ha divorziato ha dato priorità al lavoro scalando i ranghi della suddetta azienda. Grazie al suo carattere forte e nevrotico ha consolidato la sua posizione come direttrice nel marketing pubblicitario. Non è molto familiare con la tecnologia e vuole usare BugBoard26 soltanto per visualizzare lo stato del progetto e assicurarsi che tutto stia rientrando nelle scadenze.

Alessandro Romano

Age: 24

Marital status: Fidanzato

Location: Salerno, Italia

Job title: Junior Full-Stack Developer

Personal traits: Curioso, metodico, collaborativo, perfezionista

Obiettivi:

- Segnalare bug in modo chiaro e dettagliato
- Risolvere rapidamente i bug assegnati
- Collaborare con colleghi tramite commenti
- Tenere traccia della propria mole di lavoro

Interessi:

- **Gaming:** Ama giocare ai videogiochi, soprattutto gli FPS
- **Palestra:** Appassionato di bodybuilding
- **Coding:** Interessato soprattutto a JS e ai videogame engines, soprattutto Unreal Engine
- **Social:** Ama condividere le sue giornate su Instagram

Bio:

Appena assunto, laureato in Informatica. Pratico con Git, VS Code e Docker. Preferisce interfacce minimali e veloci. Alessandro è un ragazzo brillante, ama portare a spasso il cane con la ragazza e dedica il suo tempo libero nel cimentarsi nello sviluppo dei videogiochi e migliorare le sue coding skills. Vuole essere il più produttivo possibile e cerca in BugBoard26 un modo per organizzare meglio il suo lavoro. Preferisce lo smartworking.

4. Requisiti Non Funzionali e di Dominio

I requisiti non funzionali e di dominio rappresentano un complemento essenziale ai casi d'uso analizzati nel capitolo 2. Mentre i requisiti funzionali descrivono cosa il sistema deve fare, i requisiti non funzionali definiscono come il sistema deve comportarsi in termini di qualità, prestazioni e vincoli operativi.

4.1 Requisiti Non Funzionali

4.1.1 Usabilità

RNF-01: Accessibilità dell'interfaccia

- L'interfaccia web deve essere intuitiva e accessibile anche a utenti con competenze tecniche limitate, come evidenziato dalla Persona Carla Martinelli.
- Le funzionalità principali (creazione issue, visualizzazione lista, commenti) devono essere raggiungibili con massimo 3 click dalla dashboard principale.
- Il sistema deve fornire feedback visivo immediato per ogni azione dell'utente.

4.1.2 Performance

RNF-02: Tempo di risposta

- Il caricamento della lista delle issue deve avvenire in meno di 2 secondi per database contenenti fino a 500 issue.
- La creazione e il salvataggio di una nuova issue deve completarsi in meno di 3 secondi.
- Le operazioni di filtraggio e ordinamento devono essere eseguite lato client per garantire reattività immediata.

4.1.3 Sicurezza

RNF-03: Autenticazione e autorizzazione

- L'accesso al sistema deve essere protetto tramite autenticazione basata su credenziali (email e password).
- Le password devono essere memorizzate nel database in forma cifrata utilizzando algoritmi di hashing.

- Il sistema deve implementare la matrice delle autorizzazioni definita nel capitolo 2.9, garantendo che gli utenti standard possano operare solo sulle issue loro assegnate.

RNF-04: Integrità dei dati

- Il database deve garantire la consistenza dei dati attraverso transazioni ACID.
- In caso di errore durante il salvataggio, nessun dato parziale deve essere persistito nel database.

4.1.5 Manutenibilità

RNF-05: Qualità del codice

- Il codice sorgente deve seguire convenzioni di naming e formattazione standard per i linguaggi utilizzati.
- Le funzioni e i metodi critici devono essere facilmente comprensibili o commentati con spiegazioni chiare del loro funzionamento.
- Il sistema deve essere organizzato in componenti modulari con responsabilità ben definite (separazione tra logica di business, persistenza e presentazione).

RNF-06: Documentazione

- Deve essere presente un file README con istruzioni per l'installazione e l'avvio del sistema.
- Lo schema del database deve essere documentato con descrizione delle tabelle e delle relazioni.

4.1.6 Compatibilità

RNF-07: Browser supportati

- L'applicazione web deve funzionare correttamente sulle ultime versioni di Chrome, Firefox, Safari ed Edge.

RNF-08: Formati supportati

- Per l'allegato di immagini nelle issue, il sistema deve supportare tutti i formati immagine più comuni, con dimensione massima di 5MB per file.

4.2 Requisiti di Dominio

RD-01: Tracciabilità delle modifiche

- Ogni modifica effettuata su una issue (cambio stato, priorità, assegnazione) deve aggiornare automaticamente il campo "data ultima modifica".
- Ogni commento deve riportare l'autore e il timestamp di creazione.

RD-02: Classificazione obbligatoria

- Ogni issue deve essere classificata secondo una delle seguenti tipologie predefinite: Question, Bug, Documentation, Feature.
- Ogni issue deve avere uno stato tra quelli predefiniti: "todo", "in progress", "done".
- Lo stato iniziale di ogni nuova issue deve essere automaticamente impostato a "todo".

RD-03: Gestione delle priorità

- Il sistema deve supportare livelli di priorità: bassa, media, alta.
- La priorità è un attributo opzionale: se non specificata, la issue viene trattata come priorità bassa.

RD-04: Assegnazione automatica

- Ogni nuova issue deve essere automaticamente assegnata all'utente che l'ha creata.
- Solo gli amministratori possono riassegnare le issue ad altri utenti.

RD-05: Unicità delle credenziali

- Ogni indirizzo email registrato nel sistema deve essere univoco.
- Il sistema deve impedire la creazione di utenze con email già esistenti, segnalando l'errore all'amministratore.

RD-06: Persistenza dei commenti

- I commenti sono legati permanentemente alla issue di riferimento.
- L'eliminazione di una issue deve comportare l'eliminazione in cascata di tutti i commenti associati.
- I commenti devono essere visualizzati in ordine dal più recente al più vecchio.

RD-07: Validazione dei campi obbligatori

- La creazione di una issue richiede obbligatoriamente: titolo, descrizione e tipologia.
- Il titolo non può essere vuoto e deve avere una lunghezza massima di 32 caratteri.
- La descrizione non può essere vuota e deve avere una lunghezza massima di 500 caratteri.

DOCUMENTO DI SYSTEM DESIGN

BugBoard26

ID Documento	BB26-002
Versione	1.0
Stato	Rilascio Iniziale
Data di Rilascio	16/02/2026

Progetto di Ingegneria del Software – A.A. 2025/2026

Redatto da:

Antonio Esposito

Vincenzo Di Carluccio

Sommario

0. INTRODUZIONE....36
0.1 Scopo del Documento....36
0.2 Riferimenti....36
0.2 Struttura del Documento....36
1. ARCHITETTURA DEL SISTEMA....37
1.1 Stile Architetturale Scelto....37
1.2 Containerizzazione e AWS....37
1.3 Schema Architetturale....38
1.4 Motivazioni delle Scelte Tecnologiche....38
1.5 Criteri di Design....39
2. DETTAGLIO DEI SOTTOSISTEMI....40
2.1 Presentation Layer (Frontend)....40
2.2 Business Layer (Backend)....40
2.2.1 Auth Service....40
2.2.2 Core Service....41
2.3 Data Layer....41
2.3.1 Dati Strutturati (PostgreSQL)....41
2.3.2 Dati Non Strutturati (File System & Docker Volumes)....41

0. INTRODUZIONE

0.1 Scopo del Documento

Il presente documento descrive il design architetturale e le scelte tecnologiche del sistema BugBoard26, una piattaforma web per la gestione collaborativa di issue software. L'obiettivo è fornire una specifica tecnica dettagliata che guidi il team di sviluppo nell'implementazione di un sistema robusto, scalabile e manutenibile.

Questo documento si rivolge a:

- Sviluppatori che implementeranno il sistema
- Architetti software che valuteranno le scelte progettuali
- Team di testing che necessitano di comprendere l'architettura per progettare test efficaci
- Stakeholder interessati alle scelte tecnologiche

0.2 Riferimenti

Questo documento si basa sulla **Specifica dei Requisiti Software BugBoard26** (ID: BB26-001, Versione 1.0), che definisce i requisiti funzionali, non funzionali e di dominio del sistema.

0.2 Struttura del Documento

Il presente documento è organizzato nei seguenti capitoli

- **Capitolo 1 –** Architettura del sistema: verrà esposta l'architettura generale del sistema, focalizzandosi sui singoli componenti che lo compongono e il modo con cui questi comunicano.
- **Capitolo 2** – Dettaglio dei Sottosistemi – si scenderà nel dettaglio su come sono fatti i singoli sottosistemi e sulle tecnologie usate per questi ultimi.

1. ARCHITETTURA DEL SISTEMA

1.1 Stile Architetturale Scelto

Il sistema adotta un'architettura stratificata con tre livelli logici, ognuno con le proprie responsabilità:

1. **Presentation Layer (Frontend Angular):** rendering dell'interfaccia utente, gestione dello stato client-side, validazione input lato client.
2. **Business Layer (Backend Spring Boot):** implementazione della logica di business, gestione autorizzazioni, validazione dati server-side, orchestrazione delle operazioni CRUD, comunicazione col database.
3. **Data Layer (PostgreSQL)**: persistenza dati, integrità referenziale, garantisce proprietà ACID.

1.2 Containerizzazione e AWS

A supporto dei livelli logici, il sistema adotta un approccio basato sulla containerizzazione. Ogni componente software è incapsulato in un **container Docker** dedicato. Questo garantisce che l'ambiente di esecuzione sia immutabile e coerente, eliminando le problematiche di compatibilità tra l'ambiente di sviluppo e quello di produzione.

L'intera architettura è ospitata su infrastruttura cloud **Amazon Web Services (AWS)**, sfruttando un'istanza di calcolo **EC2** come nodo host. La gestione del ciclo di vita dei container e la configurazione della rete interna sono orchestrate tramite **Docker Compose**, che definisce:

- **Isolamento:** Ogni servizio gira nel proprio ambiente protetto.
- **Networking:** Una rete virtuale privata permette ai container di comunicare tra loro senza esporre porte critiche su internet.
- **Storage Persistente:** L'utilizzo di **Docker Volumes** mappati sul disco dell'istanza AWS (EBS) garantisce la persistenza dei dati del database e dei file caricati (immagini), disaccoppiando il ciclo di vita dei dati da quello dei container, che vengono cancellati nel momento in cui si stoppano.

1.3 Schema Architetturale

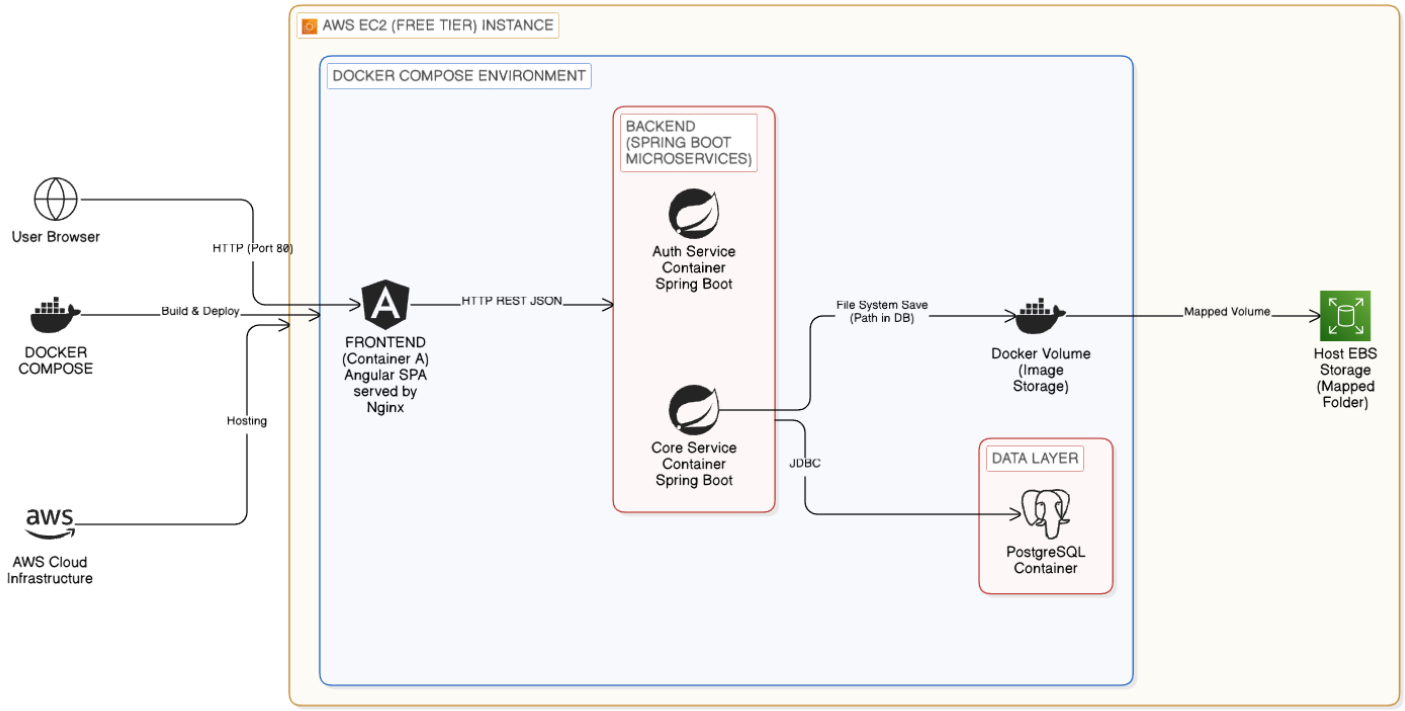


Figura 2.3 Architettura di Sistema BugBoard26

1.4 Motivazioni delle Scelte Tecnologiche

Le tecnologie selezionate rispondono a specifici requisiti di manutenibilità, consistenza e portabilità.

- **Angular (Frontend):** L'adozione del framework è motivata dalla sua architettura a componenti e dall'uso nativo di TypeScript. Questo garantisce una rigorosa tipizzazione statica, riducendo drasticamente gli errori a runtime.
- **Spring Boot (Backend):** Selezionato per la maturità del suo ecosistema. Il framework semplifica la creazione di API REST robuste.
- **PostgreSQL (Database):** Preferito per la sua affidabilità ACID e il supporto avanzato per l'integrità referenziale, essenziale per gestire le relazioni tra Utenti, Issue e commenti.
- **Docker & AWS (Infrastruttura):** La containerizzazione astrae l'applicazione dal sistema operativo sottostante, garantendo compatibilità tra gli ambienti di sviluppo e produzione. L'hosting su AWS EC2 offre un ambiente di esecuzione standardizzato e scalabile.

1.5 Criteri di Design

La progettazione di BugBoard26 è stata guidata dai seguenti principi di ingegneria del software, per garantire qualità e longevità al sistema:

- **Separation of Concerns :** Il sistema è diviso in strati (Frontend, Backend, Database) nettamente distinti. Ogni livello ha una responsabilità unica e comunica con gli altri solo attraverso interfacce ben definite (API REST), riducendo l'accoppiamento.
- **Statelessness (Backend):** I servizi Spring Boot sono progettati per essere stateless : nessuna informazione di sessione utente viene mantenuta nella memoria del server tra una richiesta e l'altra; lo stato è gestito interamente lato client (tramite Token JWT) o persistito nel database. Questo è cruciale per la scalabilità orizzontale.
- **Single Responsibility Principle (SRP):** All'interno del backend, la logica di autenticazione è separata dalla logica di business (gestione issue), facilitando la manutenzione e il testing isolato dei componenti.

2. DETTAGLIO DEI SOTTOSISTEMI

L'architettura di BugBoard26 è decomposta in moduli funzionali indipendenti, ognuno incapsulato nel proprio container Docker. Questa sezione analizza nel dettaglio le responsabilità, le tecnologie e le modalità di interazione di ciascun sottosistema.

2.1 Presentation Layer (Frontend)

Il punto di accesso per l'utente finale è costituito da una **Single Page Application** sviluppata con il framework **Angular**.

- **Responsabilità:** Il frontend gestisce interamente la logica di presentazione e la navigazione client-side. Non effettua elaborazioni di business critiche, ma agisce come un client "intelligente" che consuma le API REST esposte dal backend.
- **Hosting e Serving (Nginx):** In ambiente di produzione (containerizzato), l'applicazione non viene eseguita tramite server di sviluppo, ma viene compilata in artefatti statici (HTML, CSS, JavaScript). Questi file sono serviti da un container **Nginx**, configurato come web server leggero e ad alte prestazioni. Nginx intercetta le richieste HTTP e restituisce l'applicazione al browser.
- **Interazione:** La comunicazione con il server avviene esclusivamente tramite protocollo **HTTP**, trasmettendo dati in formato JSON.

2.2 Business Layer (Backend)

La logica applicativa risiede sul server ed è suddivisa in due servizi distinti basati su **Java Spring Boot**, per garantire una chiara separazione delle responsabilità.

2.2.1 Auth Service

Questo componente agisce come Identity Provider per il sistema.

- **Funzione:** Gestisce il ciclo di vita delle utenze (registrazione amministratori e utenti standard) e il processo di autenticazione.
- **Sicurezza:** Al momento del login, verifica le credenziali crittografate nel database e, in caso di successo, emette un token di sessione che il client utilizzerà per autenticare le richieste successive verso il Core Service.

2.2.2 Core Service

Rappresenta il motore funzionale della piattaforma BugBoard26.

- **Funzione:** Implementa i casi d'uso principali definiti nei requisiti: creazione e gestione delle Issue (Bug, Feature, Question), workflow degli stati (Todo, In Progress, Done), gestione dei commenti e assegnazione dei task.
- **Gestione File:** Questo servizio funge da interfaccia per il caricamento degli allegati. Riceve i file binari (immagini) tramite richieste e ne gestisce il salvataggio fisico, restituendo al client un riferimento (URL) per il recupero.

2.3 Data Layer

La gestione dei dati adotta un approccio ibrido per ottimizzare le prestazioni del database e la gestione dello storage.

2.3.1 Dati Strutturati (PostgreSQL)

Tutte le informazioni relazionali sono persistite in un database **PostgreSQL**.

- **Contenuto:** Tabelle per tutti gli elementi del sistema e metadati degli allegati.
- **Integrità:** Il database garantisce l'integrità referenziale e la consistenza delle transazioni (ACID).
- **Sicurezza:** Il container del database non è esposto pubblicamente; è raggiungibile esclusivamente dai container backend tramite la rete privata interna di Docker.

2.3.2 Dati Non Strutturati (File System & Docker Volumes)

Per la memorizzazione delle immagini allegate alle issue, il sistema evita l'uso di BLOB (Binary Large Objects) nel database per non degradare le prestazioni delle query.

Viene utilizzato un **Docker Volume** montato sul container del Core Service. Questo volume mappa una directory interna del container sul file system dell'istanza ospitante di AWS.

Questa configurazione garantisce che i file caricati siano persistenti e sopravvivano al riavvio o alla sostituzione dei container, simulando un object storage persistente a costo zero sull'infrastruttura di calcolo.

Documento di Design del Software

BugBoard26

ID Documento	BB26-003
Versione	1.0
Stato	Rilascio Iniziale
Data di Rilascio	27/03/2026

Progetto di Ingegneria del Software – A.A. 2025/2026

Redatto da:

Antonio Esposito

Vincenzo Di Carluccio

Sommario

0. Introduzione 46
0.1 Scopo del Documento 46
0.2 Riferimenti 46
0.3 Organizzazione del Documento 47
1. Schema di Persistenza dei Dati 48
1.1 Introduzione e Tecnologie di Persistenza 48
1.2 Diagramma del modello dei dati 49
1.3 Definizione delle Tabelle 50
1.3.1 Tabella Utenti 50
1.3.3 Tabella progetti 51
1.3.4 Tabella issue 51
1.3.5 Tabella ponte: issue_utente_assegnato 52
1.3.6 Tabella: commenti 52
1.4 Enumerazioni 53
1.5 Riepilogo delle Relazioni tra Entità 54
1.6 Considerazioni Architetturali e Vincoli di Progetto 54
2. Diagrammi delle Classi di Design 55
2.1 Introduzione 55
2.2 auth-service 55
2.3 core-service 55
2.4 Frontend Angular 55
2.5 Diagramma UML auth-service 56
2.6 Diagramma UML core-service 57
2.7 Diagramma UML Frontend 58
3. Descrizione delle scelte di Software Design 59

3.1 Architettura a microservizi 59
3.2 Pattern architetturali 59
3.2.1 Strategy Pattern 59
3.2.2 Template Method (AbstractController) 59
3.2.3 Builder Pattern 59
3.2.4 Filter Chain e Interceptor 60
3.2.5 Facade Pattern (Controller) 60
3.2.6 Chain of Responsibility (GlobalExceptionHandler) 60
3.3 Scelte di Sicurezza 61
3.3.1 JWT Stateless 61
3.3.2 Dual JWT strategy 61
3.3.3 Accesso Basato su Ruoli 61
3.3.4 BCrypt 61
3.4 Scelte di persistenza 62
3.4.1 Database condiviso con entity read-only 62
3.4.2 Enum come stringhe 62
3.4.3 FK come Integer 62
3.5 Scelte frontend 63
3.5.1 Functional guards e interceptor 63
3.5.2 LocalStorage per sessione 63
3.6 Scelte infrastrutturali 63
3.6.1 Docker Compose 63
3.6.2 Image storage su Docker volume 63
3.7 Gestione errori (RFC 7807) 64
3.8 Dependency Injection 64
3.9 Esempi di codice 64

4 Versioning 75
4.1 Sistema di Version Control 75
4.2 Strategia di Sviluppo 75
4.3 Statistiche del Repository 75
4.4 Evidenza della Cronologia Git 77
5. Report di Qualità del Codice 78
5.1 Strumento di Analisi 78
5.2 Configurazione dell'Analisi 78
5.3 Risultati dell'analisi 79

0. Introduzione

0.1 Scopo del Documento

Il presente documento costituisce il terzo artefatto della documentazione tecnica del progetto BugBoard26. Esso si propone di descrivere in modo esaustivo il design implementativo del sistema, fornendo al team di sviluppo e agli evaluatori tutti gli elementi necessari per comprendere, valutare e mantenere il codice prodotto.

Il presente documento scende al livello del software design: descrive le strutture dati persistite, le classi di design, le scelte implementative adottate e ne fornisce la motivazione tecnica.

0.2 Riferimenti

Il presente documento si basa sui seguenti artefatti prodotti nelle fasi precedenti del progetto:

ID Documento	Titolo	Descrizione	Versione
BB26-001	Specifica dei Requisiti Software	Definisce i requisiti funzionali, non funzionali e di dominio del sistema BugBoard26.	1.0
BB26-002	Documento di System Design	Descrive l'architettura del sistema, le tecnologie adottate e la struttura dei sottosistemi.	1.0

Si raccomanda la lettura preliminare di entrambi i documenti sopra citati prima di procedere con la consultazione del presente testo.

0.3 Organizzazione del Documento

Il documento è strutturato nei seguenti capitoli:

- **Capitolo 1** – Schema di Persistenza dei Dati: Descrizione del modello relazionale adottato per la persistenza dei dati nel database PostgreSQL. Sono inclusi lo schema UML, la definizione delle tabelle con i rispettivi attributi, tipi e vincoli.
- **Capitolo 2** – Diagramma delle Classi di Design: Rappresentazione UML delle classi principali del backend, con attributi, metodi e relazioni. Il diagramma riflette l'organizzazione del codice sorgente nei package Spring Boot.
- **Capitolo 3** – Scelte di Software Design: Descrizione e motivazione delle principali decisioni di design adottate durante l'implementazione. Vengono analizzati i pattern architetturali utilizzati e le motivazioni tecniche che ne hanno guidato la scelta.
- **Capitolo 4** – Versioning e Processo di Sviluppo: Evidenza dell'uso di strumenti di version control. Sono inclusi la descrizione del workflow Git adottato, statistiche aggregate (numero di commit, contributors, frequenza), e una sintesi delle pratiche collaborative seguite dal team.
- **Capitolo 5** – Report di Qualità del Codice: Risultati dell'analisi statica del codice backend generati tramite SonarQube. Vengono riportate metriche su code smells, coverage, duplicazioni e vulnerabilità.

1. Schema di Persistenza dei Dati

1.1 Introduzione e Tecnologie di Persistenza

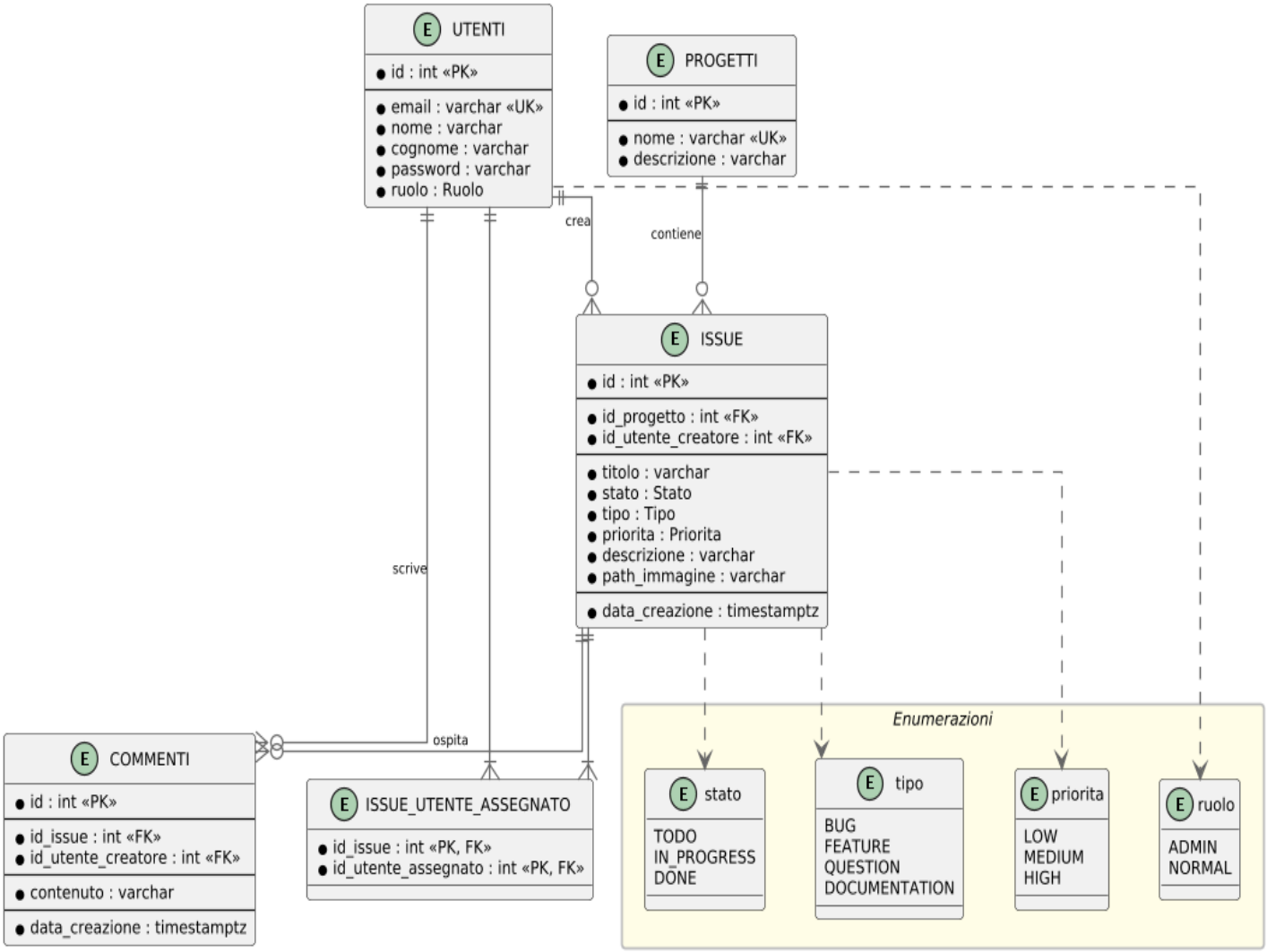
Il sistema adotta PostgreSQL 15 come DBMS principale, containerizzato tramite Docker e accessibile esclusivamente dalla rete interna Docker Compose. Il layer di persistenza Java è implementato con Spring Data JPA, che utilizza Hibernate come provider JPA sottostante.

La definizione dello schema relazionale avviene interamente in modo dichiarativo tramite le annotazioni JPA sulle entity Java: non è previsto uno script DDL manuale. Hibernate, configurato con la proprietà **spring.jpa.hibernate.ddl-auto=update**, provvede a creare o aggiornare automaticamente le tabelle all'avvio dell'applicazione, garantendo coerenza tra il modello a oggetti e lo schema fisico del database.

L'Auth Service e il Core Service condividono lo stesso database PostgreSQL ma operano su insiemi di tabelle distinti: l'Auth Service gestisce esclusivamente le tabelle di autenticazione e autorizzazione (utenti, ruoli), mentre il Core Service gestisce le tabelle di dominio applicativo (progetti, issue, commenti), anche se può comunque accedere alle tabelle degli utenti in modalità read-only.

1.2 Diagramma del modello dei dati

Il diagramma seguente illustra il modello relazionale completo del sistema. Le frecce indicano le foreign key; la tabella ISSUE_UTENTE_ASSEGNATO realizza la relazione molti-a-molti tra gli utenti e le issue a loro assegnate. Le enumerazioni (Stato, Tipo, Priorita, Ruolo) sono riportate come tipi distinti: in PostgreSQL vengono mappate come VARCHAR, mentre in JPA vengono gestite con l'annotazione @Enumerated(EnumType.STRING).



1.3 Definizione delle Tabelle

Di seguito sono descritte in dettaglio tutte le tabelle del database, con i relativi attributi, tipi di dato SQL, vincoli di integrità e la corrispondente annotazione JPA dell'entità Java.

1.3.1 Tabella Utenti

Contiene i dati anagrafici e di autenticazione degli utenti registrati nel sistema. Ogni utente è associato obbligatoriamente a un ruolo. La colonna email funge da username per Spring Security ed è soggetta a vincolo di unicità. La password è sempre memorizzata come hash BCrypt; il campo in chiaro non viene mai memorizzato.

Entità JPA corrispondente: com.bugboard26.authservice.entity.User - implementa UserDetails di Spring Security.

Colonna	Tipo SQL	Vincoli
id	SERIAL (INTEGER)	PK, NOT NULL, AUTO_INCREMENT
email	VARCHAR (100)	NOT NULL, UNIQUE
nome	VARCHAR (50)	NOT NULL
cognome	VARCHAR (50)	NOT NULL
password	VARCHAR (255)	NOT NULL
ruolo	VARCHAR (50)	NOT NULL

1.3.3 Tabella progetti

Rappresenta i progetti software tracciati dalla piattaforma. Il nome del progetto è soggetto a vincolo di unicità per evitare ambiguità. Un progetto può contenere zero o più issue.

Entità JPA corrispondente: com.bugboard26.coreservice.entity.Progetto (Core Service).

Colonna	Tipo SQL	Vincoli
id	SERIAL (INTEGER)	PK, NOT NULL, AUTO_INCREMENT
nome	VARCHAR(255)	NOT NULL, UNIQUE
descrizione	TEXT / VARCHAR	NULLABLE

1.3.4 Tabella issue

Entità centrale del sistema. Rappresenta un'attività, un bug o una richiesta di funzionalità associata a un progetto. Ogni issue registra il progetto di appartenenza, l'utente creatore, lo stato del ciclo di vita, il tipo, la priorità e un timestamp di creazione gestito automaticamente da JPA. I campi stato, tipo e priorita sono enumerazioni mappate come stringa nel database.

Entità JPA corrispondente: com.bugboard26.coreservice.entity.Issue (Core Service).

Colonna	Tipo SQL	Vincoli
id	SERIAL (INTEGER)	PK, NOT NULL, AUTO_INCREMENT
id_progetto	INTEGER	FK → progetti(id), NOT NULL
id_utente_creatore	INTEGER	FK → utenti(id), NOT NULL
titolo	VARCHAR(255)	NOT NULL
stato	VARCHAR / ENUM	NOT NULL, default TODO
tipo	VARCHAR / ENUM	NOT NULL

priorita	VARCHAR / ENUM	NOT NULL
descrizione	TEXT / VARCHAR	NULLABLE
data_creazione	TIMESTAMPTZ	NOT NULL, default NOW()
data_ultima_modifica	TIMESTAMPTZ	NOT NULL, default NOW()
url_immagine	VARCHAR	NULLABLE

1.3.5 Tabella ponte: issue_utente_assegnato

Tabella di associazione che gestisce l'assegnazione di una issue a uno o più utenti (relazione molti a molti). La chiave primaria è composta dalla coppia (id_issue, id_utente_assegnato). Permette di modellare scenari in cui più sviluppatori lavorano contemporaneamente sulla stessa issue.

Colonna	Tipo SQL	Vincoli
id_issue	INTEGER	PK, FK → issue(id), NOT NULL
id_utente_assegnato	INTEGER	PK, FK → utenti(id), NOT NULL

1.3.6 Tabella: commenti

Memorizza i commenti testuali lasciati dagli utenti su ciascuna issue. Ogni commento è collegato a una specifica issue e all'utente autore. Il timestamp di creazione è gestito automaticamente da Hibernate. La relazione con Issue è di tipo 1:N (una issue ospita zero o più commenti).

Entità JPA corrispondente: com.bugboard26.coreservice.entity.Comment (Core Service).

Colonna	Tipo SQL	Vincoli
id	SERIAL (INTEGER)	PK, NOT NULL, AUTO_INCREMENT

id_issue	INTEGER	FK → issue(id), NOT NULL
id_utente_creatore	INTEGER	FK → utenti(id), NOT NULL
contenuto	TEXT / VARCHAR	NOT NULL
data_creazione	TIMESTAMPTZ	NOT NULL, default NOW()

1.4 Enumerazioni

Il modello prevede quattro enumerazioni di dominio. In Java sono definite come enum e annotate con @Enumerated(EnumType.STRING) sulle colonne delle entity, in modo che Hibernate persista il nome letterale del valore (es. "BUG", "HIGH") anziché un indice numerico ordinale. Questa scelta garantisce maggiore leggibilità dei dati nel database e robustezza rispetto a futuri riordini dei valori enum.

Enumerazione	Valori	Entità coinvolta
Stato	TODO, IN_PROGRESS, DONE	Issue
Tipo	BUG, FEATURE, QUESTION,DOCUMENTATION	Issue
Priorita	LOW, MEDIUM, HIGH	Issue
Ruolo	ROLE_ADMIN, ROLE_UTENTE	UserRole

1.5 Riepilogo delle Relazioni tra Entità

La tabella seguente riepiloga tutte le relazioni presenti nel modello, con la cardinalità, l'eventuale tabella ponte e l'annotazione JPA utilizzata per la mappatura.

Relazione	Cardinalità	Tabella ponte	Annotazione JPA
Utente → Ruolo	N:1	—	@Enumerated su ruolo
Progetto → Issue	1:N	—	idProgetto come FK Integer
Utente → Issue (creazione)	1:N	—	idUtenteCreatore come FK Integer
Issue ↔ Utente (assegnazione)	M:N	ISSUE_UTENTE_ASSEGNATO	@ManyToMany
Issue → Commento	1:N	—	@OneToMany
Utente → Commento (autore)	1:N	—	@OneToMany

1.6 Considerazioni Architetturali e Vincoli di Progetto

- Il container PostgreSQL non espone porte verso Internet, la porta 5432 dove gira è raggiungibile soltanto dai servizi all'interno dei container. Questo garantisce un livello di isolamento sufficiente per lo scopo del progetto.
- Le immagini allegate alle issue non vengono memorizzate come BLOB nel database né su Amazon S3: sono salvate su un Docker Volume che mappa una directory del Core Service su una cartella del filesystem dell'host EC2 (EBS). Il database contiene esclusivamente il percorso del file.

2. Diagrammi delle Classi di Design

2.1 Introduzione

Questa sezione presenta i diagrammi delle classi di design del sistema in notazione UML, organizzati per microservizio. Le classi sono raggruppate per package/layer architetturale e colorate per funzione: blu per le entity, verde per i repository, arancione per i DTO, viola per i service, rosso per i controller, giallo per il layer JWT. Di seguito una breve descrizione dei principali layer dei 3 servizi.

2.2 auth-service

Il servizio è responsabile dell'autenticazione, registrazione utenti e emissione dei token JWT. I layer principali sono: entity (User, Ruolo), repository (UserRepository), DTO (LoginRequest, LoginAnswer, CreaUtenteRequest, UtenteResponse), service (AuthService con implementazione), JWT (JwtService, JwtFilter), controller (AuthController), configurazione (SecurityConfiguration, DataSeeder) e gestione eccezioni (GlobalExceptionHandler).

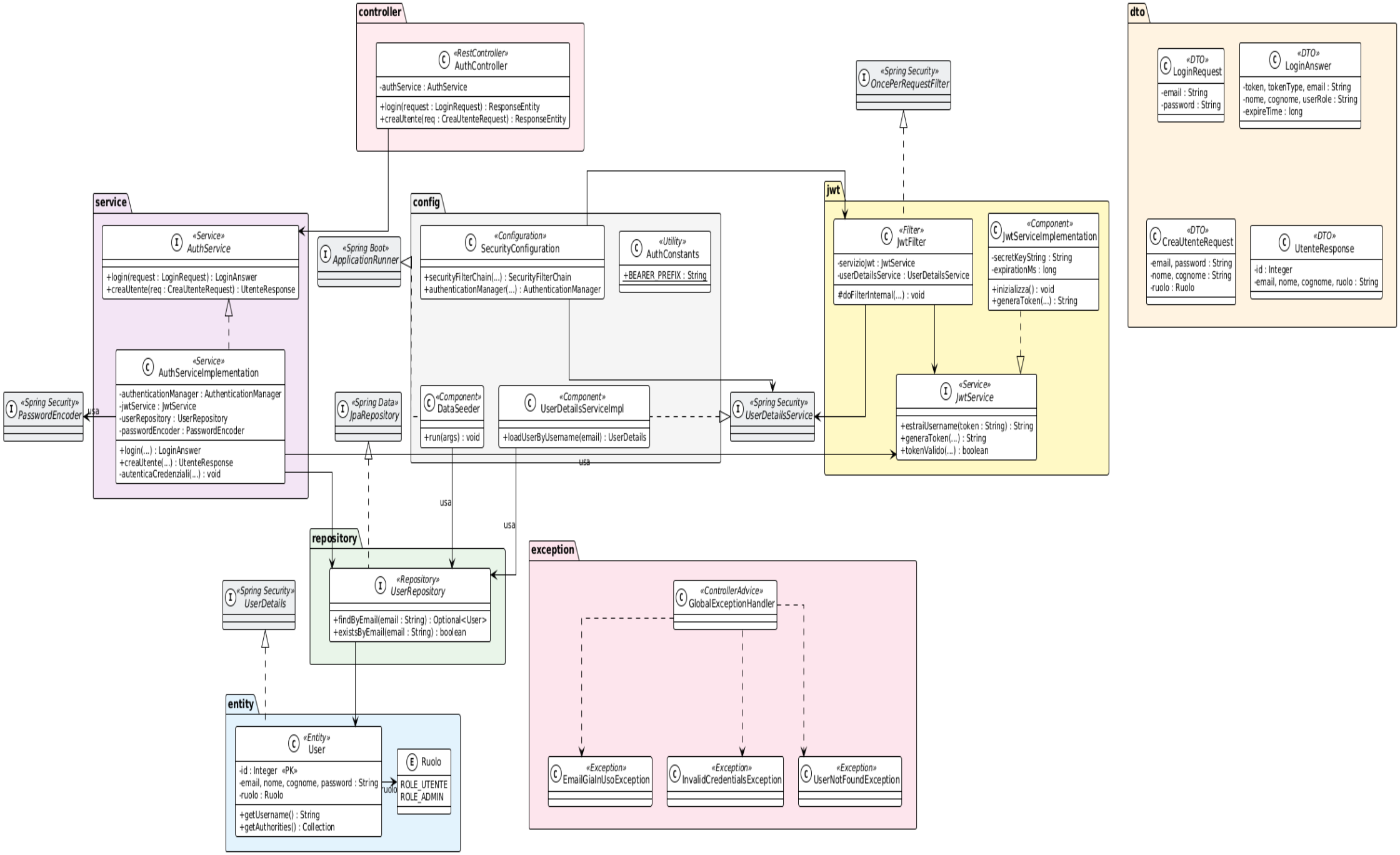
2.3 core-service

Il servizio è responsabile della gestione di progetti, issue, commenti e upload immagini. I layer principali sono: entity (Progetto, Issue, Commento, Utente read-only con @Immutable) con enumerazioni (StatoIssue, TipoIssue, PrioritaIssue), repository (uno per entita), DTO separati in request e response, service (quattro interfacce con implementazioni), controller (con AbstractController come base), JWT (validazione-only tramite JwtInterceptor), configurazione (WebConfiguration, DataSeeder) e gestione eccezioni centralizzata. La relazione chiave è il @ManyToMany tra Issue e Utente sulla join table issue_utente_assegnato.

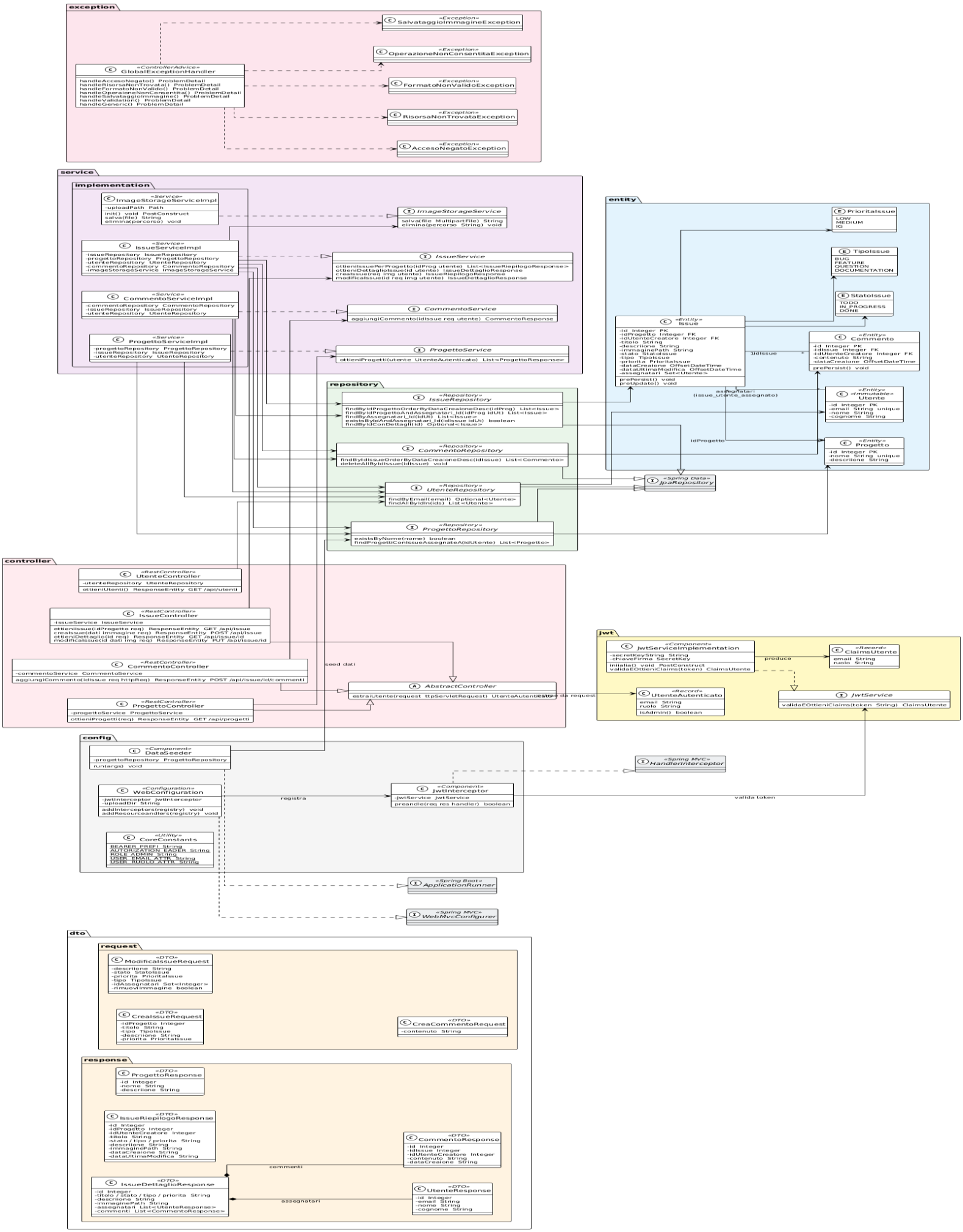
2.4 Frontend Angular

Organizzato in: models (interfacce TypeScript che mappano i DTO del backend), services (cinque servizi Injectable che consumano le API REST), guards e interceptor funzionali (authGuard, adminGuard, authInterceptor), e cinque pagine/componenti standalone (Login, IssueList, IssueCreate, IssueDetail, AdminUtenti). Ogni componente dipende dai servizi necessari per il proprio funzionamento.

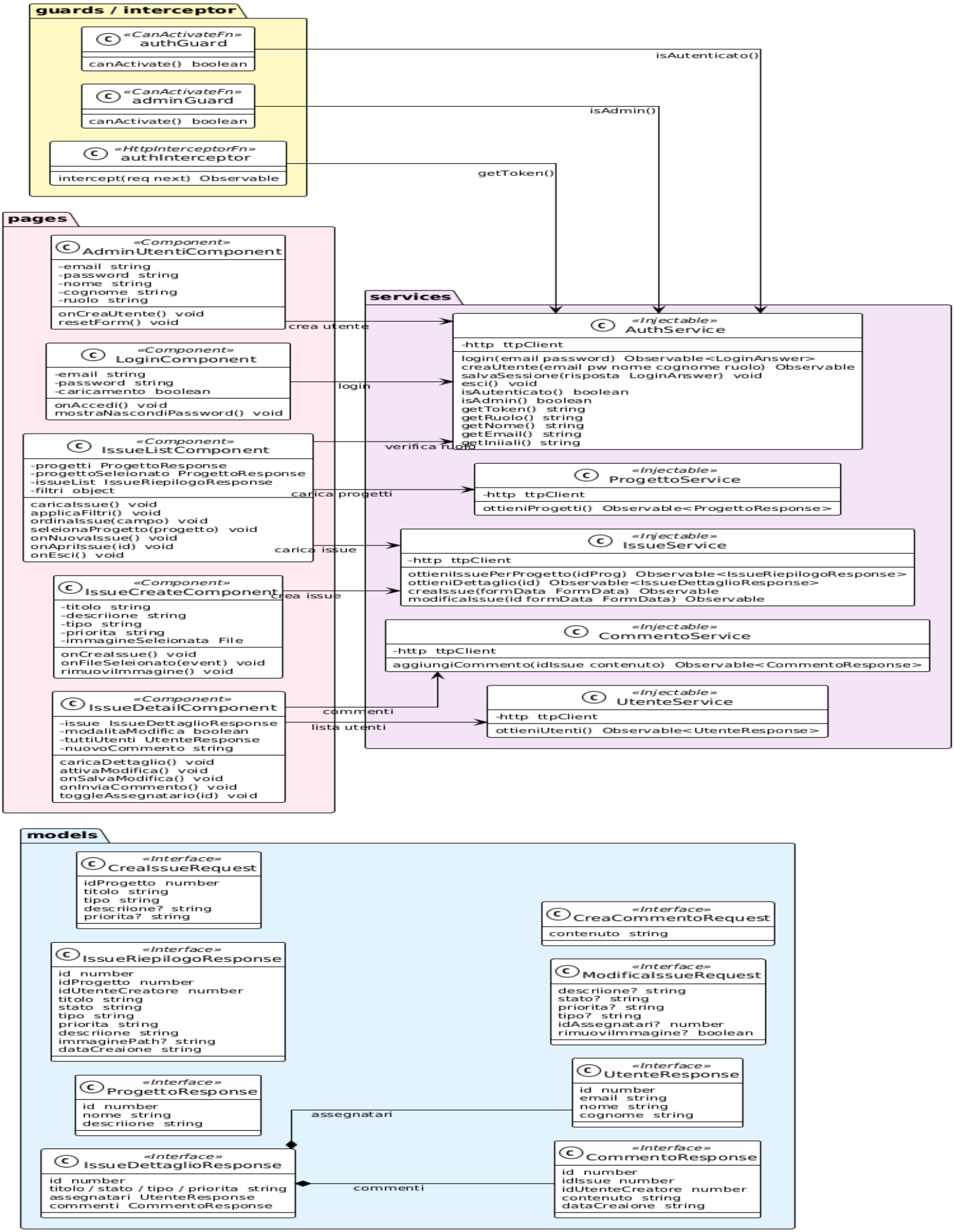
2.5 Diagramma UML auth-service



2.6 Diagramma UML core-service



2.7 Diagramma UML Frontend



3. Descrizione delle scelte di Software Design

3.1 Architettura a microservizi

Il backend è suddiviso in due microservizi distinti: auth-service e core-service. Questa scelta e motivata dal principio di Separation of Concerns: la logica di autenticazione (registrazione, login, emissione JWT) è isolata dalla logica di business (gestione progetti, issue, commenti). Ciò consente di modificare le politiche di sicurezza senza impattare il servizio core e viceversa.

I due servizi condividono lo stesso database PostgreSQL, il che semplifica notevolmente la gestione dei dati, pur sacrificando il disaccoppiamento completo tipico di un'architettura a microservizi pura. Per mitigare i rischi, il core-service accede alla tabella utenti in modalita read-only tramite l'annotazione @Immutable di Hibernate.

3.2 Pattern architetturali

3.2.1 Strategy Pattern

Tutti i servizi sono definiti come interfacce (AuthService, IssueService, CommentoService, ProgettoService, ImageStorageService, JwtService) con implementazioni concrete separate. Questo pattern garantisce:

1. **Testabilità,** poichè le dipendenze possono essere sostituite con mock nei test.
2. **Rispetto del principio Open-Closed**, permettendo di aggiungere nuove implementazioni senza modificare il codice chiamante.
3. **Dependency Inversion:** I moduli di alto livello dipendono da astrazioni e non da dettagli implementativi.

3.2.2 Template Method (AbstractController)

Nel core-service, tutti i controller (tranne UtenteController) estendono AbstractController, che fornisce il metodo protetto estraiUtente(HttpServletRequest). Questo metodo incapsula la logica di estrazione dell'identita dell'utente autenticato dagli attributi della request (impostati dal JwtInterceptor). Il pattern Template Method evita la duplicazione di questa logica comune in ogni controller, rispettando il principio DRY.

3.2.3 Builder Pattern

Le entita JPA e i DTO di risposta utilizzano l'annotazione @Builder di Lombok, che genera automaticamente il pattern Builder. Questa scelta migliora la leggibilita nella costruzione di oggetti con molti parametri (es. Issue con 12 campi, IssueDettaglioResponse) ed evita

costruttori lunghissimi. Il Builder è particolarmente utile nei metodi di mapping entity-to-DTO nei service layer.

3.2.4 Filter Chain e Interceptor

L'autenticazione JWT è implementata con due meccanismi differenti nei due servizi, riflettendo le diverse esigenze architetturali:

- **auth-service**: utilizza un JwtFilter che estende OncePerRequestFilter, integrato nella SecurityFilterChain di Spring Security. Questo approccio è necessario perchè auth-service ha bisogno dell'infrastruttura completa di Spring Security (AuthenticationManager, DaoAuthenticationProvider, BCrypt) per gestire il login e la creazione utenti.

- **core-service**: utilizza un JwtInterceptor che implementa HandlerInterceptor di Spring. Il core-service non ha dipendenza da Spring Security (non gestisce login nè password), quindi un semplice interceptor MVC è sufficiente e piu leggero. L'interceptor valida il token, estrae email e ruolo, e li imposta come attributi della request. Il path uploads/** è escluso dall'interceptor per permettere l'accesso alle immagini.

3.2.5 Controller Pattern

I controller REST espongono un'interfaccia HTTP semplificata che nasconde la complessita del service layer, dei repository e della gestione delle eccezioni. Il client interagisce con endpoint REST puliti senza conoscere i dettagli interni di validazione, autorizzazione e persistenza.

3.2.6 Chain of Responsibility (GlobalExceptionHandler)

La gestione degli errori è centralizzata in un GlobalExceptionHandler annotato con @RestControllerAdvice. Ogni tipo di eccezione e gestito da un metodo @ExceptionHandler dedicato, formando una catena di responsabilita dove Spring seleziona il gestore piu specifico. Tutte le risposte di errore seguono lo standard RFC 7807 (ProblemDetail), garantendo un formato uniforme con status, title, detail e timestamp. Questo approccio separa la logica di gestione errori dalla logica di business, evitando blocchi try-catch nei controller e nei service.

3.3 Scelte di Sicurezza

3.3.1 JWT Stateless

Il sistema adotta un'autenticazione completamente stateless basata su JWT (JSON Web Token). Il server non mantiene sessioni: ogni richiesta contiene il token nell'header Authorization (Bearer scheme), coerentemente con le linee guida REST.

3.3.2 Dual JWT strategy

Il token JWT è emesso esclusivamente dall'auth-service e validato (solo lettura) dal core-service. Entrambi i servizi condividono la stessa chiave segreta HMAC (configurata tramite variabile d'ambiente JWT_SECRET). L'auth-service ha la responsabilita completa del ciclo di vita del token (generazione, validazione, estrazione claims), mentre il core-service si limita a validare la firma e leggere i claims (email e ruolo).

3.3.3 Accesso Basato su Ruoli

Il sistema implementa un controllo accessi basato su ruoli con due ruoli: ROLE_ADMIN e ROLE_UTENTE. Il ruolo è codificato come enum Java persistito come stringa nel database e incluso come claim nel JWT. L'admin può visualizzare tutte le issue di tutti i progetti e creare nuovi utenti; l'utente vede solo le issue a cui e assegnato e i relativi progetti.

3.3.4 BCrypt

Le password sono hashate con BCrypt (tramite BCryptPasswordEncoder di Spring Security) prima della persistenza, in questo modo è garantita la sicurezza delle informazioni memorizzate.

3.4 Scelte di persistenza

3.4.1 Database condiviso con entity read-only

I due microservizi condividono lo stesso database PostgreSQL. Per garantire l'integrità dei dati, il core-service mappa la tabella utenti con l'annotazione @Immutable di Hibernate, che impedisce qualsiasi operazione di UPDATE o DELETE su quella tabella. L'entità Utente nel core-service e quindi una vista read-only dei dati gestiti dall'auth-service. Questa scelta evita la complessità di una comunicazione sincrona inter-servizio (REST call o messaging) per risolvere i dati utente.

3.4.2 Enum come stringhe

Tutti gli enum (Ruolo, StatoIssue, TipoIssue, PrioritaIssue) sono persistiti come stringhe nel database tramite @Enumerated(EnumType.STRING). Questa scelta privilegia la leggibilità dei dati nel database (i valori sono direttamente comprensibili) e la robustezza al refactoring (l'ordine degli enum può cambiare senza corrompere i dati).

3.4.3 FK come Integer

Le foreign key cross-service (es. idUtenteCreatore nelle issue e nei commenti) sono modellate come semplici Integer e non come relazioni JPA @ManyToOne. Questa scelta è dovuta al confine architetturale tra i servizi: l'entitè Utente nel core-service è read-only e non può essere target di cascade operations. L'uso di Integer come FK mantiene il disaccoppiamento e semplifica le query.

3.5 Scelte frontend

3.5.1 Functional guards e interceptor

Guards (authGuard, adminGuard) e interceptor (authInterceptor) sono implementati come funzioni (CanActivateFn, HttpInterceptorFn) anzichè come classi. Questo approccio funzionale, introdotto in Angular 14+, è piu conciso e facilmente testabile rispetto all'approccio class-based tradizionale.

3.5.2 LocalStorage per sessione

I dati di sessione (token JWT, ruolo, nome, cognome, email) sono persistiti in localStorage del browser. Questa scelta è coerente con l'architettura stateless: il server non mantiene sessioni, quindi il client deve conservare il token per le richieste successive. Il logout consiste nella semplice rimozione dei dati da localStorage (client-side).

3.6 Scelte infrastrutturali

3.6.1 Docker Compose

L'orchestrazione dei container avviene tramite Docker Compose, che permette di definire e avviare tutti i servizi con un singolo comando. Ogni servizio ha un Dockerfile dedicato con build multi-stage (builder Maven + runtime JRE Alpine) per minimizzare la dimensione delle immagini. Tutti i container eseguono con utente non-root (appuser) per sicurezza.

3.6.2 Image storage su Docker volume

Le immagini allegate alle issue sono salvate su un Docker volume (core_uploads) mappato alla directory /uploads del container core-service. Il database memorizza solo il percorso relativo (es. /api/uploads/uuid.png). Questa scelta evita:

1. L'uso di servizi cloud a pagamento come S3.
2. Il salvataggio di BLOB nel database, che degraderebbe le performance, anche se per un Progetto di queste dimensioni non cambierebbe quasi nulla.
3. La necessita di configurazioni cloud aggiuntive. I file sono serviti direttamente da Spring tramite ResourceHandler configurato in WebConfiguration.

3.7 Gestione errori (RFC 7807)

Entrambi i servizi backend adottano lo standard RFC 7807 "Problem Details for HTTP APIs" per le risposte di errore, utilizzando la classe ProblemDetail di Spring. Ogni risposta di errore include: status HTTP, title (categoria dell'errore), detail (messaggio specifico) e timestamp. Questo formato standardizzato facilita la gestione degli errori lato client e garantisce uniformità nelle risposte di errore dell'intera API.

3.8 Dependency Injection

La dependency injection è implementata esclusivamente tramite constructor injection, utilizzando l'annotazione @RequiredArgsConstructor di Lombok.

3.9 Esempi di codice

Di seguito vengono presentati alcune immagini di parti significative del codice sorgente.

```
You, last week | 2 authors (You and one other)
public interface IssueService {

    /**
     * Issue di un progetto al click sul nome del progetto.
     * Admin: tutte le issue del progetto.
     * Utente: solo le issue a lui assegnate in quel progetto.
     */
    List<IssueRiepilogoResponse> ottieniIssuePerProgetto(Integer idProgetto, UtenteAutenticato utenteCorrente);

    /**
     * Crea una nuova issue nel progetto specificato.
     * Il creatore viene aggiunto automaticamente agli assegnatari.
     * Stato iniziale: TO-DO. Priorità iniziale: LOW.
     * @param immagine file opzionale da allegare alla issue
     */
    IssueRiepilogoResponse creaIssue(CreaIssueRequest request, MultipartFile immagine, UtenteAutenticato utenteCorrente);

    /**
     * Dettaglio di una issue con assegnatari e commenti.
     * Admin: accesso libero. Utente: solo se assegnato.
     */
    IssueDettaglioResponse ottieniDettaglioIssue(Integer idIssue, UtenteAutenticato utenteCorrente);

    /**
     * Aggiorna una issue esistente.
     * Utente: tutti i campi, ma solo delle issue a lui assegnate.
     * Admin: tutti i campi di qualsiasi issue.
     * @param immagine nuova immagine opzionale; se null lascia invariata quella esistente
     */
    IssueRiepilogoResponse modificaIssue(Integer idIssue, ModificaIssueRequest request, MultipartFile immagine, UtenteAutenticato utenteCorrente);
}
```

Figura 4.1: Strategy Pattern – Interfaccia IssueService

```java
public class IssueServiceImplementation implements IssueService
{
    private final IssueRepository issueRepository;
    private final ProgettoRepository progettoRepository;
    private final UtenteRepository utenteRepository;
    private final CommentoRepository commentoRepository;
    private final ImageStorageService imageStorageService;

    @Override
    @Transactional(readOnly = true)
    public List<IssueRiepilogoResponse> ottieniIssuePerProgetto(Integer idProgetto, UtenteAutenticato utenteCorrente)
    {
        verificaEsistenzaProgetto(idProgetto);
        Utente utente = trovaUtentePerEmail(utenteCorrente.email());

        return trovaIssuePerProgetto(idProgetto, utente, utenteCorrente)
                .stream().map(this::mappaIssueRiepilogo).toList();
    }

    @Override
    public IssueRiepilogoResponse creaIssue(CreaIssueRequest request, MultipartFile immagine, UtenteAutenticato utenteCorrente)
    {
        Utente utente = trovaUtentePerEmail(utenteCorrente.email());
        verificaEsistenzaProgetto(request.getIdProgetto());

        HashSet<Utente> assegnatari = new HashSet<>();
        assegnatari.add(utente);

        String immaginePath = (immagine != null && !immagine.isEmpty()) ? imageStorageService.salva(immagine) : null;

        Issue nuovaIssue = Issue.builder()
                .idProgetto(request.getIdProgetto())
                .idUtenteCreatore(utente.getId())
                .titolo(request.getTitolo())
                .tipo(request.getTipo())
                .stato(StatoIssue.TODO)
                .priorita(request.getPriorita() != null ? request.getPriorita() : PrioritaIssue.LOW)
                .descrizione(request.getDescrizione())
                .immaginePath(immaginePath)
                .assegnatari(assegnatari)
                .build();

        Issue salvata = issueRepository.save(nuovaIssue);
        log.info(format: "Issue creata: id={}, progetto={}, creatore={}", salvata.getId(), request.getIdProgetto(), utente.getId());
        return mappaIssueRiepilogo(salvata);
    }
```

Figura 4.2: Strategy Pattern e Builder Pattern – IssueServiceImplementation con Issue.builder()

```java
package com.bugboard26.coreservice.controller;        Antonio PC portatile, 2 weeks ago •

import com.bugboard26.coreservice.constants.CoreConstants;
import com.bugboard26.coreservice.jwt.UtenteAutenticato;
import jakarta.servlet.http.HttpServletRequest;

Antonio PC portatile, 2 weeks ago | 1 author (Antonio PC portatile)
/**
 * Classe base che fornisce ai controller il metodo per estrarre
 * l'utente autenticato dagli attributi impostati da JwtInterceptor.
 */
public abstract class AbstractController
{
    protected UtenteAutenticato estraiUtente(HttpServletRequest request)
    {
        String email = (String) request.getAttribute(CoreConstants.USER_EMAIL_ATTR);
        String ruolo = (String) request.getAttribute(CoreConstants.USER_RUOLO_ATTR);
        return new UtenteAutenticato(email, ruolo);
    }
}
```

Figura 4.3: Template Method – AbstractController con il metodo estraiUtente()

```java
@Entity
@Table(name = "issue")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Issue
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "id_progetto", nullable = false)
    private Integer idProgetto;

    @Column(name = "id_utente_creatore", nullable = false)
    private Integer idUtenteCreatore;

    @Column(nullable = false)
    private String titolo;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private StatoIssue stato;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private TipoIssue tipo;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private PrioritaIssue priorita;

    @Column
    private String descrizione;
```

Figura 4.4: Entity Issue con @Builder, @Enumerated(EnumType.STRING)

```java
@Slf4j
@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private final JwtService servizioJwt;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
    throws ServletException, IOException
    {
        final String jwt = estraiTokenDaHeader(request);
        // Se c'è un token e l'utente non è già autenticato nel contesto
        if (jwt != null && SecurityContextHolder.getContext().getAuthentication() == null) processaAutenticazioneToken(jwt, request);
        filterChain.doFilter(request, response);
    }

    /**
     * Estrae la stringa JWT dall'header Authorization se presente e ben formattata.
     */
    private String estraiTokenDaHeader(HttpServletRequest request)
    {
        final String authorizationHeader = request.getHeader(AuthenticationConstants.AUTHORIZATION_HEADER);
        if (authorizationHeader == null || !authorizationHeader.startsWith(AuthenticationConstants.BEARER_PREFIX)) return null;

        return authorizationHeader.substring(AuthenticationConstants.BEARER_PREFIX.length());
    }

    /**
     * Gestisce il ciclo di vita della validazione e autenticazione del token.
     */
    private void processaAutenticazioneToken(String jwt, HttpServletRequest request)
    {
        try
        {
            final String userEmail = servizioJwt.estraiUsername(jwt);
            if (userEmail != null) autenticaUtente(userEmail, jwt, request);
        }
        catch (JwtException | IllegalArgumentException e)
        {
            // Log dettagliato per facilitare il debug di problemi con i token
            log.warn(format: "Tentativo di autenticazione fallito con token non valido: {}", e.getMessage());
        }
    }

    /**
     * Carica i dettagli utente, verifica il token e aggiorna il SecurityContext.
     */
    private void autenticaUtente(String userEmail, String jwt, HttpServletRequest request)
    {
        UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);
```

Figura 4.5: Filter Chain – JwtFilter (auth-service) estende OncePerRequestFilter

```java
@Slf4j
@Component
@RequiredArgsConstructor
public class JwtInterceptor implements HandlerInterceptor
{
    private final JwtService jwtService;

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) throws Exception
    {
        String token = estraiToken(request);

        if (token == null)
        {
            log.warn(format: "Richiesta rifiutata: token assente [{}]", request.getRequestURI());
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED, msg: "Token JWT assente");
            return false;
        }

        try
        {
            ClaimsUtente claims = jwtService.validaEOttieniClaims(token);
            request.setAttribute(CoreConstants.USER_EMAIL_ATTR, claims.email());
            request.setAttribute(CoreConstants.USER_RUOLO_ATTR, claims.ruolo());
            return true;
        }
        catch (Exception e)
        {
            log.warn(format: "Richiesta rifiutata: token non valido [{}] — {}", request.getRequestURI(), e.getMessage());
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED, msg: "Token JWT non valido o scaduto");
            return false;
        }
    }

    private String estraiToken(HttpServletRequest request)
    {
        String header = request.getHeader(CoreConstants.AUTHORIZATION_HEADER);
        if (header == null || !header.startsWith(CoreConstants.BEARER_PREFIX)) return null;
        return header.substring(CoreConstants.BEARER_PREFIX.length());
    }
}
```

Figura 4.6: Interceptor – JwtInterceptor (core-service) implementa HandlerInterceptor

```java
@RestController
@RequestMapping("/api/issue")
@RequiredArgsConstructor
public class IssueController extends AbstractController
{
    private final IssueService issueService;

    @GetMapping
    public ResponseEntity<List<IssueRiepilogoResponse>> ottieniIssuePerProgetto(
            @RequestParam Integer idProgetto,
            HttpServletRequest request)
    {
        return ResponseEntity.ok(issueService.ottieniIssuePerProgetto(idProgetto, estraiUtente(request)));
    }

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<IssueRiepilogoResponse> creaIssue(
            @RequestPart("dati") @Valid CreaIssueRequest body,
            @RequestPart(value = "immagine", required = false) MultipartFile immagine,
            HttpServletRequest request)
    {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(issueService.creaIssue(body, immagine, estraiUtente(request)));
    }

    @GetMapping("/{id}")
    public ResponseEntity<IssueDettaglioResponse> ottieniDettaglio(
            @PathVariable Integer id,
            HttpServletRequest request)
    {
        return ResponseEntity.ok(issueService.ottieniDettaglioIssue(id, estraiUtente(request)));
    }

    @PutMapping(value = "/{id}", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<IssueRiepilogoResponse> modificaIssue(
            @PathVariable Integer id,
            @RequestPart("dati") @Valid ModificaIssueRequest body,
            @RequestPart(value = "immagine", required = false) MultipartFile immagine,
            HttpServletRequest request)
    {
        return ResponseEntity.ok(issueService.modificaIssue(id, body, immagine, estraiUtente(request)));
    }
}
```

Figura 4.7: IssueController espone endpoint REST delegando al service

```java
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler
{
    @ExceptionHandler(AccesoNegatoException.class)
    public ProblemDetail handleAccesoNegato(AccesoNegatoException e)
    {
        log.warn(format: "Accesso negato: {}", e.getMessage());
        return creaProblemDetail(HttpStatus.FORBIDDEN, title: "Accesso negato", e.getMessage());
    }

    @ExceptionHandler(RisorsaNonTrovataException.class)
    public ProblemDetail handleRisorsaNonTrovata(RisorsaNonTrovataException e)
    {
        log.warn(format: "Risorsa non trovata: {}", e.getMessage());
        return creaProblemDetail(HttpStatus.NOT_FOUND, title: "Risorsa non trovata", e.getMessage());
    }

    @ExceptionHandler(FormatoNonValidoException.class)
    public ProblemDetail handleFormatoNonValido(FormatoNonValidoException e)
    {
        log.warn(format: "Formato non valido: {}", e.getMessage());
        return creaProblemDetail(HttpStatus.BAD_REQUEST, title: "Formato non valido", e.getMessage());
    }

    @ExceptionHandler(OperazioneNonConsentitaException.class)
    public ProblemDetail handleOperazioneNonConsentita(OperazioneNonConsentitaException e)
    {
        log.warn(format: "Operazione non consentita: {}", e.getMessage());
        return creaProblemDetail(HttpStatus.CONFLICT, title: "Operazione non consentita", e.getMessage());
    }

    @ExceptionHandler(SalvataggioImmagineException.class)
    public ProblemDetail handleSalvataggioImmagine(SalvataggioImmagineException e)
    {
        log.error(format: "Errore salvataggio immagine: {}", e.getMessage(), e);
        return creaProblemDetail(HttpStatus.INTERNAL_SERVER_ERROR, title: "Errore salvataggio immagine", e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidazione(MethodArgumentNotValidException e)
    {
        String dettagli = e.getBindingResult().getFieldErrors().stream()
                .map(fe -> fe.getField() + ": " + fe.getDefaultMessage())
                .collect(Collectors.joining(delimiter: "; "));
        log.warn(format: "Errore di validazione: {}", dettagli);
        return creaProblemDetail(HttpStatus.BAD_REQUEST, title: "Errore di validazione", dettagli);
    }
```

Figura 4.8: Chain of Responsibility – GlobalExceptionHandler con @RestControllerAdvice

```java
public class SecurityConfiguration
{
    private final UserDetailsService userDetailsService;

    /**
     * Catena di filtri di sicurezza principale.
     */
    @Bean
    @SuppressWarnings("java:S4502")
    public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtFilter filtroJwt)
    {
        return http
            // Disabilitiamo CSRF perché stiamo usando JWT
            .csrf(csrf -> csrf.disable())
            // Configuriamo le regole di autorizzazione
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(AuthenticationConstants.PUBLIC_PATH).permitAll()
                .requestMatchers(HttpMethod.POST, AuthenticationConstants.ADMIN_USERS_PATH).hasAuthority(AuthenticationConstants.ROLE_ADMIN)
                .anyRequest().authenticated()
            )
            // Configuriamo la gestione delle sessioni come stateless
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            // Configuriamo il provider di autenticazione personalizzato
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(filtroJwt, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    /**
     * Provider che gestisce l'autenticazione tramite database.
     * Usa BCrypt per verificare le password.
     */
    @Bean
    public AuthenticationProvider authenticationProvider()
    {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    /**
     * AuthenticationManager esposto come Bean per poterlo iniettare
     * nei servizi (es. AuthServiceImpl per validare login).
     */
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
    {
        return config.getAuthenticationManager();
    }
```

Figura 4.9: SecurityConfiguration – SecurityFilterChain, BCrypt, SessionCreationPolicy.STATELESS

```java
@Entity
@Table(name = "utenti")
@Immutable
@Getter
@NoArgsConstructor
public class Utente
{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String nome;

    @Column(nullable = false)
    private String cognome;

    @Override
    public boolean equals(Object o)
    {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Utente utente = (Utente) o;
        return id != null && Objects.equals(id, utente.id);
    }

    @Override
    public int hashCode() { return getClass().hashCode(); }
}
```

Figura 4.10: Entity @Immutable – Utente read-only nel core-service

```java
public interface IssueRepository extends JpaRepository<Issue, Integer> {

    Show AOT-generated Implementation, Query, etc...
    /** Admin – tutte le issue di un progetto, ordinate per data di creazione discendente. */
    List<Issue> findByIdProgettoOrderByDataCreazioneDesc(Integer idProgetto);

    Show AOT-generated Implementation, Query, etc...
    /** Utente normale – issue di un progetto specifico assegnate all'utente. */
    List<Issue> findByIdProgettoAndAssegnatari_IdOrderByDataCreazioneDesc(Integer idProgetto, Integer idUtente);

    Show AOT-generated Implementation, Query, etc...
    /** Bulk – tutte le issue assegnate a un utente su tutti i progetti. */
    List<Issue> findByAssegnatari_Id(Integer idUtente);

    Show AOT-generated Implementation, Query, etc...
    /** Verifica se una issue è assegnata a un utente (controllo permessi UC-05, UC-06, UC-07). */
    boolean existsByIdAndAssegnatari_Id(Integer id, Integer idUtente);

    Show AOT-generated Implementation, Query, etc...
    /** Verifica se un utente ha almeno una issue assegnata in un progetto (controllo visibilità progetto). */
    boolean existsByIdProgettoAndAssegnatari_Id(Integer idProgetto, Integer idUtente);

    Show AOT-generated Implementation, Query, etc...
    /** Dettaglio issue con assegnatari in un'unica query per evitare N+1 (UC-07). */
    @Query("SELECT i FROM Issue i LEFT JOIN FETCH i.assegnatari WHERE i.id = :issueId")
    Optional<Issue> findByIdConDettagli(@Param("issueId") Integer issueId);
}
```

Figura 4.11: IssueRepository – Derived queries e @Query con LEFT JOIN FETCH

```gitignore
import { inject } from '@angular/core';        Vinsent034, last week • feat(auth): cr
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

// Guard funzionale che protegge le rotte riservate agli utenti autenticati
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isAutenticato()) {
    return true;
  }

  // Utente non autenticato: reindirizza al login
  return router.createUrlTree(['/login']);
};
```

Figura 4.12: Functional Guard Angular – authGuard con CanActivateFn

```java
@Data
public class CreaIssueRequest
{
    @NotNull
    private Integer idProgetto;

    @NotBlank
    @Size(max = 32)
    private String titolo;

    @NotNull
    private TipoIssue tipo;

    @Size(max = 500)
    private String descrizione;

    /** Se null, il service applica il default LOW. */
    private PrioritaIssue priorita;
}
```

Figura 4.13: DTO con validazione – CreaIssueRequest (@NotNull, @NotBlank, @Size)

Quelli riportati sopra sono soltanto alcuni esempi dei dettagli implementativi del codice sorgente del sistema. Per visualizzare il codice in formato integrale, si prega di far riferimento alla repository Github discussa nel capitolo successivo.

4 Versioning

4.1 Sistema di Version Control

Il progetto BugBoard26 utilizza Git come sistema di version control distribuito, ospitato su un repository GitHub. La scelta di Git è motivata dalla sua diffusione universale nell'ecosistema di sviluppo software e dalla sua totale integrazione con i principali ambienti di sviluppo.

Il repository, visionabile qui, è organizzato come monorepo e contiene al suo interno tutto il codice sorgente del sistema.

4.2 Strategia di Sviluppo

Il team ha adottato una strategia di branching semplificata, coerente con le dimensioni ridotte del gruppo di lavoro. Tutto lo sviluppo avviene direttamente sul branch main, che rappresenta in ogni momento lo stato funzionante e stabile del sistema.

La politica adottata prevede una regola fondamentale: ogni commit sul branch main deve corrispondere a codice verificato e funzionante. Prima di effettuare un push, lo sviluppatore è tenuto a validare le proprie modifiche, assicurandosi che la compilazione vada a buon fine, che i servizi si avviino correttamente e che il codice funzioni come previsto. Questo approccio garantisce che il branch main rimanga sempre in uno stato funzionante.

4.3 Statistiche del Repository

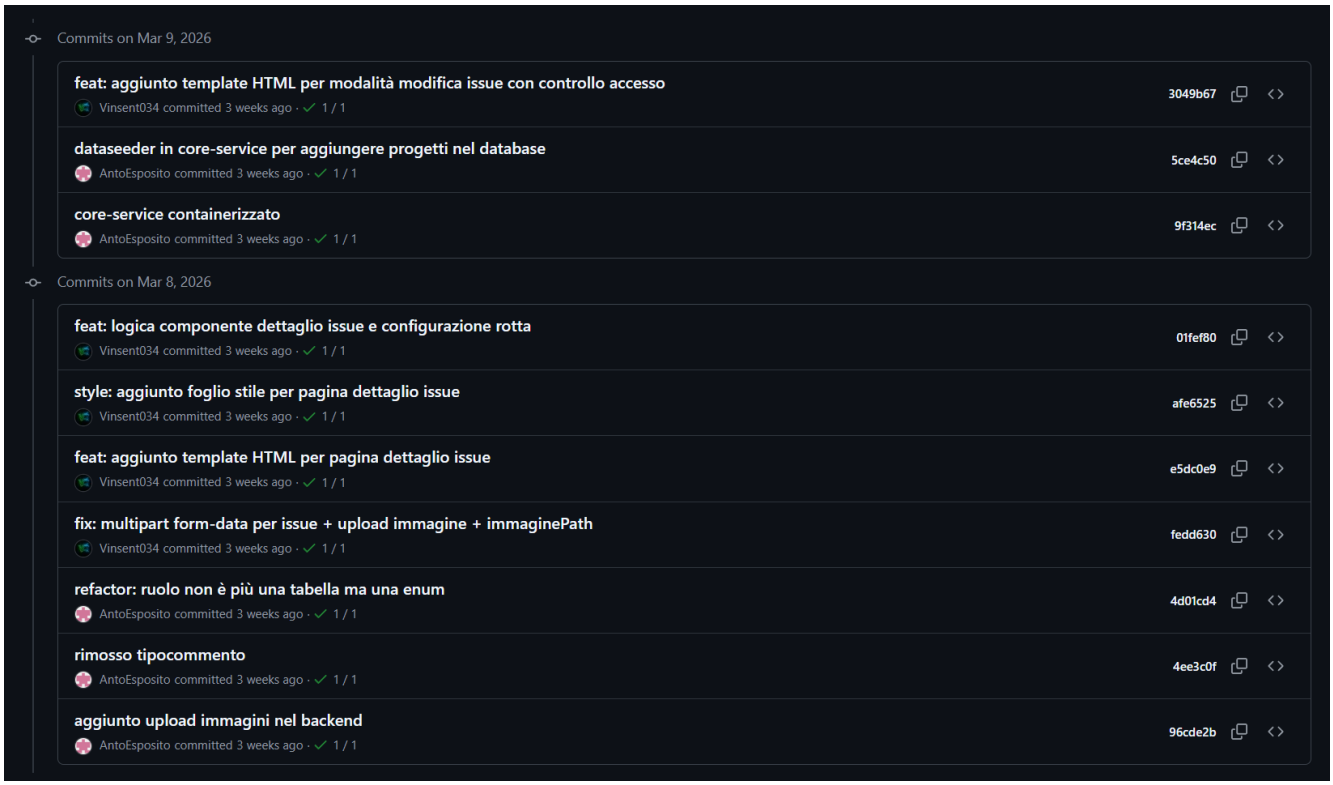
Di seguito vengono riportate alcune statistiche estratte dal repository GitHub del progetto.

Commits
Number of commits per week
Commits
60
50
40
30
20
10
0
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Jan
Feb
Mar
Apr

Contributors
Contributions per week to main, excluding merge commits
Period: All
Contributions: Commits
Commits over time
Weekly from 8 feb 2026 to 29 mar 2026
Contributions
50
40
30
20
10
0
9 Feb
16 Feb
23 Feb
2 Mar
9 Mar
16 Mar
23 Mar
30 Mar
AntoEsposito
100 commits 8364 ++ 3200 --
#1
Contributions
20
10
0
16 Feb
2 Mar
16 Mar
30 Mar
Vinsent034
76 commits 13.755 ++ 563 --
#2
Contributions
20
10
0
16 Feb
2 Mar
16 Mar
30 Mar

4.4 Evidenza della Cronologia Git

Di seguito viene riportato un estratto della cronologia dei commit sul branch main.



5. Report di Qualità del Codice

5.1 Strumento di Analisi

Per la verifica della qualità del codice sorgente è stato utilizzato SonarQube, uno degli strumenti di analisi statica più diffusi nell'industria del software. SonarQube ispeziona il codice senza eseguirlo, individuando problematiche quali code smell, bug potenziali, vulnerabilità di sicurezza e duplicazioni.

L'analisi è stata eseguita sui due microservizi backend del sistema (auth-service e core-service), che costituiscono la componente più critica dal punto di vista della logica di business e della sicurezza. Il frontend Angular, pur non analizzato con SonarQube, beneficia della tipizzazione statica offerta da TypeScript che mitiga una parte significativa delle problematiche rilevabili tramite analisi statica.

È significativo menzionare che è stato anche fattro uso dell'apposita estensione ufficiale SonarLint per gli ambienti di sviluppo, che ha permesso un'analisi dinamica del codice sorgente, evitando di generare warning ed issue ancor prima di pushare il codice nella repository.

5.2 Configurazione dell'Analisi

L'istanza SonarQube è stata eseguita tramite GitHub Actions, il quale ad ogni push esegue una compilazione e scansione del codice e fornisce una visualizzazione dei risultati sul sito ufficiale SonarCloud.io.

L'analisi è configurata esclusivamente sul codice backend. Il workflow esegue tre fasi principali: compilazione dei sorgenti tramite Maven, risoluzione delle dipendenze, e infine la scansione statica attraverso l'action ufficiale SonarSource/sonarqube-scan-action. Il token di autenticazione verso SonarCloud è memorizzato come repository secret di GitHub, garantendo che le credenziali non siano esposte nel codice sorgente.

Il progetto su SonarCloud è visualizzabile a questo link.

5.3 Risultati dell'analisi

La tabella seguente riepiloga i risultati dell'analisi per il backend.

Metrica	Valore	Rating
Security Issues	0	A
Reliability Issues	0	A
Maintainability Issues	0	A
Duplications	0.8%	-
Lines of Code	2731	-

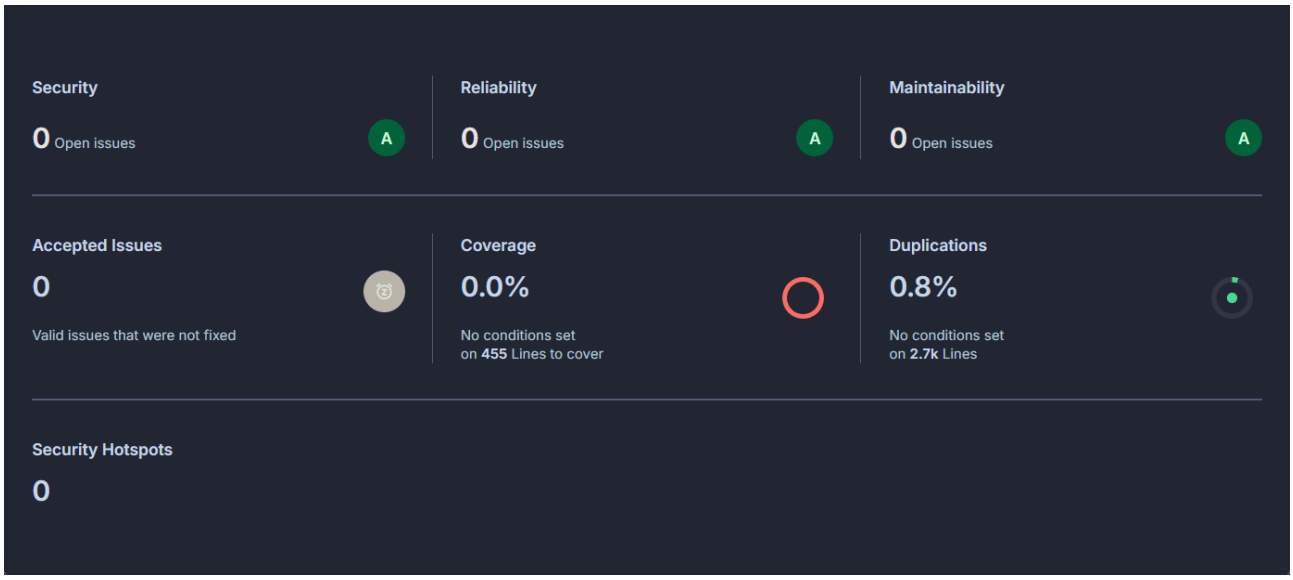


Figura 6.3 – SonarQubeCloud Analysis Summary

È Importante sottolineare che dati i requisiti di testing imposti dalla traccia del progetto (testing di soli 4 metodi) sarebbe stato impossibile raggiungere una Coverage soddisfacente per far passare il quality Gate di SonarQube (80%). Pertanto, si è deciso di non includere la test coverage nell'analisi tramite JaCoCo.

Documento di Software Testing

BugBoard26

ID Documento	BB26-004
Versione	1.0
Stato	Rilascio Iniziale
Data di Rilascio	03/04/2026

Progetto di Ingegneria del Software – A.A. 2025/2026

Redatto da:

Antonio Esposito

Vincenzo Di Carluccio

Sommario

1. Test Plan 82
1.1 Oggetto del test 82
1.2 Approccio 82
1.3 Ambiente di test 82
1.4 Riepilogo test case 83
2. Strategie di progettazione dei test 84
2.1 Input Space Partitioning Functionality-Based 84
2.2 Classi di equivalenza individuate 84
2.3 Criterio di copertura: WECT (Each Choice Coverage) 87
2.4 Strategie non adottate e motivazioni 87
3. Codice di Testing 88

1. Test Plan

1.1 Oggetto del test

Il test plan riguarda la funzionalita di Gestione Issue del microservizio core-service di BugBoard26. Tale funzionalita comprende quattro operazioni principali esposte dal service layer:

- **modificaIssue**: modifica di una issue esistente (descrizione, stato, priorita, assegnatari, immagine allegata);
- **creaIssue**: creazione di una nuova issue all'interno di un progetto;
- **ottieniIssuePerProgetto**: recupero della lista di issue di un progetto, filtrata in base al ruolo dell'utente autenticato.
- **aggiungiCommento:** consente di aggiungere un commento a una issue esistente.

1.2 Approccio

L'approccio adottato è il black-box testing basato su Input Space Partitioning (ISP). Per ogni metodo sotto test, i parametri di ingresso sono stati analizzati e suddivisi in classi di equivalenza derivanti dalle regole di business del dominio. Il criterio di copertura scelto è il N-WECT (Weak Normal Equivalence Class Testing), anche noto come Each Choice Coverage: per ogni classe di equivalenza di ogni parametro viene selezionato almeno un test case rappresentativo.

1.3 Ambiente di test

Componente	Dettaglio
Linguaggio	Java 25
Framework di test	JUnit 5
Mocking	Mockito 5 + MockitoExtension
Build tool	Maven
IDE	VS Code (Windows 11)

1.4 Riepilogo test case

Metodo	Parametri	Test Case	Classi coperte
modificaIssue	4	11	Admin/utente assegnato/non assegnato, issue esistente/inesistente, gestione assegnatari, upload/rimozione/sostituzione immagine, request vuota
creaIssue	3	4	Progetto esistente/inesistente, con/senza immagine, priorita null/esplicita, utente inesistente
ottieniIssuePerProgetto	2	4	Admin (tutte)/utente (solo assegnate), progetto vuoto, progetto inesistente
aggiungiCommento	3	5	Admin/utente assegnato/non assegnato, issue inesistente, utente inesistente

2. Strategie di progettazione dei test

2.1 Input Space Partitioning Functionality-Based

La strategia di progettazione dei test adottata è l'Input Space Partitioning (ISP) con approccio functionality-based. L'input space di ciascun metodo viene partizionato in classi di equivalenza derivanti dalle specifiche funzionali e dalle regole di business dell'applicazione, piuttosto che dalla struttura interna del codice.

Per ogni parametro del metodo sotto test, si identificano le classi di equivalenza significative. L'ipotesi di base e che tutti i valori appartenenti alla stessa classe producano lo stesso comportamento del sistema; pertanto, un singolo rappresentante per classe e sufficiente.

2.2 Classi di equivalenza individuate

modificaIssue(Integer idIssue, ModificaIssueRequest request, MultipartFile immagine, UtenteAutenticato utente)

Parametro	Classe di equivalenza	Valore rappresentativo	Test Case
idIssue	Issue esistente nel DB	1	TC-01..02, 04..11
idIssue	Issue inesistente	999	TC-03
request.descrizione	Presente (non null)	"Nuova descrizione aggiornata"	TC-01
request.descrizione	Assente (null)	null	TC-02, TC-11
request.stato	Nuovo stato valido	IN_PROGRESS	TC-02
request.stato	Assente (null)	null	TC-01, TC-11
request.priorita	Nuova priorita valida	HIGH	TC-02
request.priorita	Assente (null)	null	TC-01, TC-11
request.idAssegnatari	Set di ID validi	Set{2, 3}	TC-05
request.idAssegnatari	Set con ID inesistente	Set{2, 999}	TC-07
request.idAssegnatari	Null (non fornito)	null	TC-01, TC-02
request.rimuoviImmagine	true	true	TC-09, TC-10

request.rimuovilmmagine	false (default)	false	TC-01..08, TC-11
immagine	File presente (non vuoto)	mock(MultipartFile)	TC-08, TC-10
immagine	Null (nessun upload)	null	TC-01..07, TC-09, TC-11
utente	ROLE_ADMIN	admin@test.com	TC-01, TC-05, TC-07..11
utente	ROLE_UTENTE assegnato	utente@test.com	TC-02, TC-06
utente	ROLE_UTENTE non assegnato	estraneo@test.com	TC-04

crealssue(CrealssueRequest request, MultipartFile immagine, UtenteAutenticato utente)

Parametro	Classe di equivalenza	Valore rappresentativo	Test Case
request.idProgetto	Progetto esistente	10	TC-12, TC-13
request.idProgetto	Progetto inesistente	999	TC-14
request.priorita	Null (default LOW)	null	TC-12
request.priorita	Valore esplicito	HIGH	TC-13
immagine	File presente	mock(MultipartFile)	TC-13
immagine	Null	null	TC-12, TC-14, TC-15
utente	Utente esistente nel DB	utente@test.com	TC-12..14
utente	Utente inesistente nel DB	utente@test.com (no DB)	TC-15

ottieniIssuePerProgetto(Integer idProgetto, UtenteAutenticato utente)

Parametro	Classe di equivalenza	Valore rappresentativo	Test Case
idProgetto	Progetto esistente con issue	10	TC-16, TC-17
idProgetto	Progetto esistente senza issue	10	TC-18
idProgetto	Progetto inesistente	999	TC-19
utente	ROLE_ADMIN (vede tutte)	admin@test.com	TC-16, TC-18, TC-19
utente	ROLE_UTENTE (solo assegnate)	utente@test.com	TC-17

aggiungiCommento(Integer idIssue, CreaCommentoRequest request, UtenteAutenticato utente)

Parametro	Classe di equivalenza	Valore rappresentativo	Test Case
idIssue	Issue esistente	10	TC-01..03
idIssue	Issue inesistente	999	TC-04
request.contenuto	Stringa valida	"Commento di test"	TC-01..05
utente	ROLE_ADMIN	admin@test.com	TC-01, TC-04
utente	ROLE_UTENTE assegnato	utente@test.com	TC-02
utente	ROLE_UTENTE non assegnato	utente@test.com (non assegnato)	TC-03
utente	Email inesistente nel DB	fantasma@test.com	TC-05

2.3 Criterio di copertura: WECT (Each Choice Coverage)

Il criterio di copertura adottato e il Weak Normal Equivalence Class Testing (WECT), noto anche come Each Choice Coverage. Questo criterio richiede che ogni classe di equivalenza di ogni parametro sia coperta da almeno un test case, senza imporre vincoli sulle combinazioni tra classi di parametri diversi.

La scelta del WECT e motivata dal bilanciamento tra copertura e costo: il numero di test case cresce linearmente con il numero di classi (anziche esponenzialmente come in All-Combinations/SECT), garantendo comunque che ogni partizione rilevante sia esercitata almeno una volta.

In totale sono stati prodotti 24 test case che coprono tutte le classi di equivalenza individuate per i 4 metodi sotto test.

2.4 Strategie non adottate e motivazioni

Strategia	Descrizione	Motivazione
Boundary Value Analysis	Test ai confini delle classi (min, min+, max-, max).	I parametri sotto test sono prevalentemente oggetti, enum e booleani, non valori numerici continui con confini significativi.
SECT (All Combinations)	Test per tutte le combinazioni di classi di equivalenza.	Crescita esponenziale dei test case (es. modificaIssue con 18 classi produrrebbe centinaia di combinazioni) senza proporzionale aumento della probabilita di trovare difetti.
R-WECT / R-SECT (Robust)	Estensione con valori fuori dominio (null, stringhe vuote, etc.).	La validazione degli input e delegata a Bean Validation (@NotBlank, @NotNull) a livello controller; il service layer riceve dati gia validati.
Pair-Wise / T-Wise	Copertura delle interazioni tra coppie di parametri.	Non si sono individuate interazioni significative tra parametri che non siano gia catturate dalle classi di equivalenza funzionali.
White-Box Testing	Copertura strutturale (statement, branch, condition, MC/DC).	Richiede strumenti di coverage (JaCoCo) e accesso al codice sorgente durante la progettazione dei test; l'approccio scelto e puramente black-box, coerente con i requisiti del progetto.

3. Codice di Testing

Le immagini seguenti mostrano la struttura e l'organizzazione dei test nell'IDE, evidenziando i test case, le classi helper e i risultati di esecuzione. Per il codice completo si prega di far riferimento al repository GitHub.

```java
@ExtendWith(MockitoExtension.class)
class CommentoServiceImplementationTest
{
    @Mock private CommentoRepository commentoRepository;
    @Mock private IssueRepository issueRepository;
    @Mock private UtenteRepository utenteRepository;
    @InjectMocks private CommentoServiceImplementation commentoService;

    private static final UtenteAutenticato ADMIN =
            new UtenteAutenticato(email: "admin@test.com", ruolo: "ROLE_ADMIN");
    private static final UtenteAutenticato UTENTE =
            new UtenteAutenticato(email: "utente@test.com", ruolo: "ROLE_UTENTE");

    // ── Helper ──────────────────────────────────────────────────────────────   You, 5 da

    // Utente e' @Immutable (no setter, no builder), quindi usiamo
    // ReflectionTestUtils di Spring per impostare i campi privati.
    private Utente creaUtente(Integer id, String email, String nome, String cognome)
    {
        Utente u = new Utente();
        ReflectionTestUtils.setField(u, "id", id);
        ReflectionTestUtils.setField(u, "email", email);
        ReflectionTestUtils.setField(u, "nome", nome);
        ReflectionTestUtils.setField(u, "cognome", cognome);
        return u;
    }

    private CreaCommentoRequest creaRequest(String contenuto)
    {
        CreaCommentoRequest req = new CreaCommentoRequest();
        req.setContenuto(contenuto);
        return req;
    }
```

Figura 1 — Setup della classe CommentoServiceImplementationTest

```java
// — Test Case 01: Admin aggiunge commento ————————————————————————

@Test
@DisplayName("Test Case 01: aggiungiCommento – admin, issue esistente")
void aggiungiCommento_adminIssueEsistente()
{
    Utente utenteAdmin = creaUtente(id: 1, email: "admin@test.com", nome: "Admin", cognome: "User");
    when(utenteRepository.findByEmail(email: "admin@test.com")).thenReturn(Optional.of(utenteAdmin));
    when(issueRepository.existsById(10)).thenReturn(true);

    Commento commentoSalvato = Commento.builder()
            .id(id: 100)
            .idIssue(idIssue: 10)
            .idUtenteCreatore(idUtenteCreatore: 1)
            .contenuto(contenuto: "Commento di test")
            .dataCreazione(OffsetDateTime.now())
            .build();
    when(commentoRepository.save(any(Commento.class))).thenReturn(commentoSalvato);

    CreaCommentoRequest request = creaRequest(contenuto: "Commento di test");

    CommentoResponse response = commentoService.aggiungiCommento(idIssue: 10, request, ADMIN);      You, 5 days

    assertThat(response.getId()).isEqualTo(expected: 100);
    assertThat(response.getIdIssue()).isEqualTo(expected: 10);
    assertThat(response.getIdUtenteCreatore()).isEqualTo(expected: 1);
    assertThat(response.getContenuto()).isEqualTo(expected: "Commento di test");
    assertThat(response.getDataCreazione()).isNotNull();
    verify(commentoRepository).save(any(Commento.class));
}
```

Figura 2 — Test case TC-01: admin aggiunge commento su issue esistente.

```java
// — Test Case 03: Utente non assegnato ————————————————————————

@Test
@DisplayName("Test Case 03: aggiungiCommento – utente non assegnato")
void aggiungiCommento_utenteNonAssegnato()
{
    Utente utente = creaUtente(id: 2, email: "utente@test.com", nome: "Mario", cognome: "Rossi");
    when(utenteRepository.findByEmail(email: "utente@test.com")).thenReturn(Optional.of(utente));
    when(issueRepository.existsById(10)).thenReturn(true);
    when(issueRepository.existsByIdAndAssegnatari_Id(id: 10, idUtente: 2)).thenReturn(false);

    CreaCommentoRequest request = creaRequest(contenuto: "Tentativo non autorizzato");      You, 5 day

    assertThatThrownBy(() -> commentoService.aggiungiCommento(idIssue: 10, request, UTENTE))
            .isInstanceOf(type: AccesoNegatoException.class);
    verify(commentoRepository, never()).save(any());
}
```

Figura 3 — Test case TC-03: utente non assegnato, verifica AccesoNegatoException.

```java
// — Test Case 08: Upload nuova immagine ————————————————————————

@Test
@DisplayName("Test Case 08: modificaIssue – upload nuova immagine, vecchia eliminata")
void modificaIssue_uploadNuovaImmagine()
{
    Issue issue = creaIssueEsistente();
    when(issueRepository.findById(1)).thenReturn(Optional.of(issue));
    when(issueRepository.save(any(Issue.class))).thenReturn(issue);

    MultipartFile nuovaImmagine = mock(MultipartFile.class);
    when(nuovaImmagine.isEmpty()).thenReturn(false);
    when(imageStorageService.salva(nuovaImmagine)).thenReturn("/api/uploads/new-uuid.png");

    ModificaIssueRequest request = new ModificaIssueRequest();

    IssueRiepilogoResponse response = issueService.modificaIssue(idIssue: 1, request, nuovaImmagine, ADMIN);

    verify(imageStorageService).elimina(percorsoRelativo: "/api/uploads/old.jpg");
    verify(imageStorageService).salva(nuovaImmagine);
    assertThat(response.getImmaginePath()).isEqualTo(expected: "/api/uploads/new-uuid.png");
}
```

Figura 4 — Test case TC-08: upload nuova immagine con eliminazione della vecchia.

```java
// — Test Case 12: creaIssue – progetto esistente, no immagine, priorita null –

@Test
@DisplayName("Test Case 12: creaIssue – progetto esistente, no immagine, priorita null")
void creaIssue_progettoEsistenteSenzaImmaginePrioritaNull()
{
    Utente utente = creaUtente(id: 1, email: "utente@test.com", nome: "Utente", cognome: "Test");
    when(utenteRepository.findByEmail(email: "utente@test.com")).thenReturn(Optional.of(utente));
    when(progettoRepository.existsById(10)).thenReturn(true);

    CreaIssueRequest request = new CreaIssueRequest();
    request.setIdProgetto(idProgetto: 10);
    request.setTitolo(titolo: "Bug critico");
    request.setTipo(TipoIssue.BUG);
    request.setDescrizione(descrizione: "Descrizione del bug");
    request.setPriorita(priorita: null);

    ArgumentCaptor<Issue> issueCaptor = ArgumentCaptor.forClass(Issue.class);
    Issue issueSalvata = Issue.builder()
            .id(id: 100)
            .idProgetto(idProgetto: 10)
            .titolo(titolo: "Bug critico")
            .stato(StatoIssue.TODO)        Vinsent034, yesterday • aggiunta test
            .tipo(TipoIssue.BUG)
            .priorita(PrioritaIssue.LOW)
            .descrizione(descrizione: "Descrizione del bug")
            .assegnatari(new HashSet<>(Set.of(utente)))
            .build();
    when(issueRepository.save(any(Issue.class))).thenReturn(issueSalvata);

    IssueRiepilogoResponse response = issueService.creaIssue(request, immagine: null, UTENTE);

    assertThat(response.getStato()).isEqualTo(StatoIssue.TODO);
    assertThat(response.getPriorita()).isEqualTo(PrioritaIssue.LOW);

    verify(issueRepository).save(issueCaptor.capture());
    assertThat(issueCaptor.getValue().getAssegnatari()).contains(utente);

    verify(imageStorageService, never()).salva(any());
}

// — Test Case 13: creaIssue – con immagine e priorita=HIGH ————————
```

Figura 5 — Test case TC-12: creaIssue con progetto esistente, priorita null e default LOW.